

Embedding-basierte Enterprise-Systeme: Theoretische Grundlagen und Architektur moderner AI-Use-Cases

Maria Pape

Betreuung: Prof. Dr. Simon Hirländer

in Zusammenarbeit mit SPAR ICS

2025-11-04



Paris-Lodron-Universität Salzburg

BSc. Artificial Intelligence

Zusammenfassung

Unternehmen verfügen über große Mengen strukturierter und unstrukturierter Daten, deren semantische Nutzung für KI-Anwendungen eine zentrale Herausforderung darstellt. Diese Arbeit untersucht, welche Embedding-basierten Methoden sich zur semantischen Integration unternehmensspezifischen Wissens in KI-Systeme eignen und wie darauf aufbauende Business-Use-Cases anhand einer gemeinsamen Architektur realisiert werden können. Dazu werden zunächst klassische Wort-Embeddings, kontextualisierte Transformer-Repräsentationen sowie Satz- und Dokument-Embeddings theoretisch eingeordnet. Anschließend wird eine gemeinsame Grundarchitektur Embedding-basierter Enterprise-Systeme abgeleitet. Diese umfasst Datenaufbereitung, Embedding-Erzeugung, Speicherung in einem Vektorindex, Ähnlichkeitsberechnung, Retrieval sowie die anwendungsspezifische Weiterverarbeitung der Ergebnisse. Die Architektur wird anhand dreier prototypischer Use-Cases im Retail-Kontext veranschaulicht: einer semantischen Produktsuche, eines Retrieval-Augmented-Generation-basierten Produkt-Chatbots sowie eines Empfehlungssystems auf Basis von Collaborative Filtering und Embedding-Matching. Die Resultate zeigen, dass sich insbesondere Satz- und Dokument-Embeddings für semantische Suche und RAG-Anwendungen eignen, während Empfehlungssysteme zusätzlich von Item-, User- und Interaktions-Embeddings profitieren. Gleichzeitig wird deutlich, dass Embeddings allein keine vollständige Lösung darstellen. Die Qualität solcher Systeme hängt wesentlich von Datenrepräsentation, Modellwahl, Retrieval-Qualität, Rankinglogik und Evaluierung ab. Die Arbeit zeigt damit, dass Embeddings als verbindendes Architekturelement moderner Enterprise-KI-Systeme verstanden werden können, deren praktischer Nutzen erst durch die Einbettung in eine robuste, domänenspezifisch angepasste Gesamtarchitektur entsteht.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung: Warum sind Embeddings zentral für KI in Unternehmen?	1
1.2	Zielsetzung und methodisches Vorgehen	1
2	Theoretische und technische Grundlagen	3
2.1	Natural Language Processing (NLP)	3
2.2	Semantische Repräsentation durch Embeddings	3
2.2.1	Vektorräume als Grundlage	4
2.2.2	Definition von Embeddings	4
2.2.3	Verteilungsansatz	5
2.2.4	Ähnlichkeitsmaße im semantischen Raum	5
2.3	Wort-Embeddings und zentrale Modelle	6
2.3.1	Ko-Okkurrenz-Matrix	6
2.3.2	Zählbasierte Modelle	7
2.3.3	Vorhersagebasierte Modelle	8
2.3.3.1	Word2vec	9
2.3.3.2	Skip-Gram	9
2.3.3.3	GloVe	9
3	Moderne Embeddings	10
3.1	Von statischen zu kontextualisierten Repräsentationen	10
3.2	Transformer als Grundlage kontextualisierter Embeddings	11
3.2.1	Self-Attention	13
3.3	Kontextualisierte Embeddings	14
3.3.1	BERT	14
3.3.2	GPT	16
3.3.3	Ähnlichkeitsmaße für kontextualisierte Embeddings	16
3.4	Satz- und Dokument-Embeddings	17
3.4.1	SBERT	18
4	Gemeinsame Grundarchitektur Embedding-basierter Enterprise-Systeme	19

4.1	Datenaufbereitung	19
4.2	Embedding-Erzeugung und Speicherung	20
4.3	Embedding-Retrieval	21
4.3.1	Bi-Encoder und Cross-Encoder	21
4.4	Retrieval-Augmented Generation und generative Sprachmodelle	22
4.5	Embedding-Matching und Ranking	24
4.5.1	Collaborative Filtering	24
4.6	Optimierung und Evaluierung	25
4.7	Grenzen von Embeddings	27
4.8	Pipeline Embedding-basierter Enterprise-Systeme	28
5	Business Use-Cases anhand von Embeddings	29
5.1	Anwendungsbeispiel: Semantische Produktsuche im Enterprise-Kontext . .	31
5.1.1	Methodik	31
5.1.2	Interpretation der Ergebnisse	32
5.2	Anwendungsbeispiel: Enterprise Wissen in Chatbots durch RAG	34
5.2.1	Methodik	34
5.2.2	Interpretation der Ergebnisse	35
5.3	Anwendungsbeispiel: Retail-Empfehlungssystem	37
5.3.1	Methodik	37
5.3.2	Interpretation der Ergebnisse	39
5.4	Kritische Einordnung der betrachteten Use-Cases	41
6	Fazit	43
6.1	Beantwortung der Forschungsfrage	43
6.2	Kritische Reflexion und Grenzen der Arbeit	44
6.3	Ausblick und zukünftige Forschungsansätze	45

1 Einleitung

1.1 Problemstellung: Warum sind Embeddings zentral für KI in Unternehmen?

Im Zuge der zunehmenden Digitalisierung und der wachsenden Verfügbarkeit unstrukturierter Daten gewinnt der Einsatz Künstlicher Intelligenz (KI) im Enterprise-Kontext kontinuierlich an Bedeutung. Unternehmen verfügen heute über große Mengen an Informationen in Form von Dokumenten, E-Mails, Wissensdatenbanken, Produktdaten oder Nutzerinteraktionen. Eine zentrale Herausforderung besteht darin, diese Daten nicht nur zu speichern, sondern semantisch zugänglich und für KI-Systeme nutzbar zu machen. Insbesondere Anwendungen wie semantische Suchen, Retrieval-Augmented Generation (RAG) oder personalisierte Empfehlungssysteme erfordern Verfahren, die Bedeutungen, Zusammenhänge und Kontextinformationen maschinell verarbeiten können. Eine Schlüsseltechnologie zur Lösung dieses Problems stellen sogenannte Embeddings dar. Sie ermöglichen es, Wörter, Sätze, Dokumente oder andere Datenobjekte in numerische Vektorrepräsentationen zu überführen und semantische Beziehungen innerhalb eines Vektorraums abzubilden. Dadurch können KI-Systeme inhaltliche Ähnlichkeiten erkennen und Informationen kontextabhängig verarbeiten. Embeddings bilden heute die Grundlage zahlreicher moderner Anwendungen im Bereich des Natural Language Processing sowie vieler Enterprise-Softwares. Während unterschiedliche Embedding-basierte Verfahren häufig isoliert betrachtet werden, zeigt sich bei näherer Analyse, dass viele dieser Systeme trotz unterschiedlicher Zielsetzungen einer gemeinsamen technischen Grundarchitektur folgen.

1.2 Zielsetzung und methodisches Vorgehen

In dieser Ausarbeitung wird folgende Forschungsfrage behandelt:

„Welche Embedding-basierten Methoden eignen sich zur semantischen Integration unternehmensspezifischen Wissens in KI-Systeme, und wie lassen sich darauf aufbauende Business-Use-Cases anhand einer gemeinsamen Architektur realisieren?“

Ziel dieser Arbeit ist es daher, die theoretischen und technischen Grundlagen von Em-

beddings zu erläutern und darauf aufbauend anhand von prototypischen Beispielen zu analysieren, wie unterschiedliche Embedding-basierte Enterprise-Use-Cases auf derselben Architektur aufbauen.

Das methodische Vorgehen gliedert sich in drei Schritte. Zunächst werden die theoretischen und technischen Grundlagen von Embeddings literaturbasiert aufgearbeitet. Dabei werden klassische Wort-Embeddings, zählbasierte und vorhersagebasierte Verfahren sowie moderne kontextualisierte Transformer-basierte Repräsentationen betrachtet. Ergänzend werden zentrale Konzepte wie Vektorräume, semantische Ähnlichkeit, Kosinus-Ähnlichkeit, Satz- und Dokument-Embeddings sowie Retrieval-Verfahren erläutert. Diese Grundlagen bilden die Basis für das Verständnis moderner Enterprise-Anwendungen, die auf semantischer Repräsentation und Ähnlichkeitsberechnung beruhen. Im zweiten Schritt wird aus der zuvor beschriebenen theoretischen Basis eine gemeinsame Grundarchitektur Embedding-basierter Enterprise-Systeme abgeleitet. Diese Architektur beschreibt die zentralen Verarbeitungsschritte, die in unterschiedlichen Anwendungsszenarien wiederkehren: Datenaufbereitung, Erzeugung von Vektorrepräsentationen, Speicherung in einem Vektorindex, Berechnung semantischer Ähnlichkeiten sowie die anwendungsspezifische Weiterverarbeitung der Ergebnisse. Die Architektur wird dabei als abstrahiertes Referenzmodell verstanden, das unabhängig vom konkreten Use-Case eingesetzt und angepasst werden kann. Drittens wird diese Architektur anhand prototypischer Business-Use-Cases im Retail-Kontext veranschaulicht. Betrachtet werden eine semantische Produktsuche, ein Retrieval-Augmented-Generation-basierter Produkt-Chatbot sowie ein Empfehlungssystem auf Basis von Collaborative Filtering und Embedding-Matching. Die Implementierungen dienen als Proof of Concept und sollen zeigen, wie die zuvor entwickelte Grundarchitektur praktisch angewendet werden kann. Sie sind nicht als vollständig optimierte Produktivsysteme zu verstehen, sondern als exemplarische Umsetzungen zur Analyse der gemeinsamen technischen Struktur Embedding-basierter Anwendungen. Die Bewertung der Use-Cases erfolgt qualitativ. Untersucht wird insbesondere, welche Rolle Embeddings innerhalb der jeweiligen Anwendung übernehmen, welche Verarbeitungsschritte wieder verwendet werden können und welche Grenzen sich aus Modellwahl, Datenrepräsentation und Retrieval-Qualität ergeben. Eine systematische quantitative Evaluation verschiedener Embedding-Modelle, ein domänenspezifisches Fine-Tuning sowie ein Benchmarking anhand großer Testdatensätze sind nicht Gegenstand dieser Arbeit. Diese Abgrenzung

ist notwendig, da der Fokus auf der konzeptionellen Einordnung und architektonischen Vergleichbarkeit der betrachteten Systeme liegt.

Der wissenschaftliche Beitrag dieser Arbeit besteht somit in der systematischen Einordnung zentraler Embedding-basierter Verfahren für Anwendungsszenarien in Unternehmen, der Ableitung einer gemeinsamen semantischen Grundarchitektur sowie der prototypischen Plausibilisierung dieser Architektur anhand dreier praxisnaher Business-Use-Cases. Dadurch soll gezeigt werden, dass semantische Suche, RAG-basierte Chatbots und Empfehlungssysteme trotz unterschiedlicher Zielsetzungen auf gemeinsamen Mechanismen der semantischen Repräsentation und Ähnlichkeitsberechnung aufbauen.

2 Theoretische und technische Grundlagen

2.1 Natural Language Processing (NLP)

Natural Language Processing (NLP) beschreibt die Schnittstelle zwischen Linguistik und Informatik und beschäftigt sich mit der maschinellen Verarbeitung natürlicher Sprache. Für Embedding-Methoden spielt insbesondere die Semantik eine zentrale Rolle, da sie sich mit der Bedeutung von Wörtern und sprachlichen Zusammenhängen beschäftigt. Ziel moderner NLP-Modelle ist es, Sprache in semantische Repräsentationen zu überführen, die maschinell verarbeitet und für nachgelagerte Anwendungen genutzt werden können. Solche Repräsentationen bilden die Grundlage moderner Embedding-Verfahren und vieler KI-Anwendungen im Enterprise-Kontext ([Camacho-Collados and Pilehvar 2020](#)).

2.2 Semantische Repräsentation durch Embeddings

Um das Grundproblem semantischer Repräsentation zu verstehen, kann das Wort „Tisch“ als Beispiel dienen. Ein Computer verarbeitet Informationen ausschließlich in Form von Nullen und Einsen, weshalb Buchstaben zunächst in Bitfolgen umgewandelt werden müssen. In einer einfachen Zeichenkodierung wie ASCII werden jedem Zeichen 8 Bit zugeordnet. Das Wort „Tisch“ ergibt somit eine Sequenz aus 40 Bits. Diese Bit-Repräsentation hat jedoch einen entscheidenden Nachteil: Die semantische Bedeutung geht vollständig verloren. Für den Computer bestehen keinerlei Zusammenhänge zwischen den Bitmustern der Wörter „Tisch“, „Tafel“ oder „Anrichte“, obwohl diese inhaltlich eng verwandt

sind. Damit Maschinen semantische Ähnlichkeiten erkennen können, muss die Repräsentation eines Wortes Informationen über seine Bedeutung enthalten. Eine der wichtigsten Lösungen für dieses Problem sind Embeddings, also eingebettete Repräsentationen im Vektorraum. Sie ermöglichen, dass Wörter, Sätze, Dokumente oder sogar Bilder in Form kompakter, numerischer Vektoren dargestellt werden, die semantische Beziehungen abbilden ([Camacho-Collados and Pilehvar 2020](#)).

2.2.1 Vektorräume als Grundlage

Ein Vektor ist eine geordnete Sequenz numerischer Werte und stellt ein Element eines Vektorraums dar. Ein Vektorraum wiederum ist eine Menge solcher Vektoren und wird durch seine Dimension beschrieben. Ein dreidimensionaler Raum enthält beispielsweise Vektoren, die drei Koordinatenwerte besitzen.

Das sogenannte Vektorraum-Modell, welches ursprünglich von [Salton et al. \(1975\)](#) vorgeschlagen wurde, bietet eine elegante Lösung für die limitierte Bitrepräsentation. Wörter oder Dokumente werden darin als Punkte in einem kontinuierlichen, mehrdimensionalen Raum abgebildet. Dieser Raum wird im NLP auch semantischer Raum genannt, da er semantische Relationen zwischen den eingebetteten Objekten widerspiegelt ([Russell and Norvig 2009](#); [Salton et al. 1975](#); [Jurafsky and Martin 2025](#)).

2.2.2 Definition von Embeddings

Ein Embedding ist eine maschinenlesbare, numerische Repräsentation, die die Bedeutung eines Wortes, Satzes oder Dokuments einfängt. Die Grundidee besteht darin, jedes Objekt in Form eines Vektors in einem hochdimensionalen Raum darzustellen. Dabei wird ein Vokabular der Größe m in einen n -dimensionalen Raum projiziert, wobei in der Regel $n \leq m$ gilt. Dieser wird auch als semantischer Raum bezeichnet. Ein semantischer Raum entsteht aus den Verteilungen der kontextuellen Nachbarn eines Wortes, also aus den Umgebungen, in denen das Wort typischerweise vorkommt. Wörter mit ähnlicher Bedeutung liegen in diesem Raum näher beieinander, da ihre Vektoren ähnliche Richtungen und Längen aufweisen. Dadurch wird ein mathematisch fundiertes Ähnlichkeitsmaß zwischen Begriffen möglich. Die Vektorraum-Methode zählt zu den erfolgreichsten und am besten untersuchten Ansätzen im Natural Language Processing und bildet die Grundlage moderner Modelle zur semantischen Repräsentation ([Camacho-Collados and Pilehvar 2020](#);

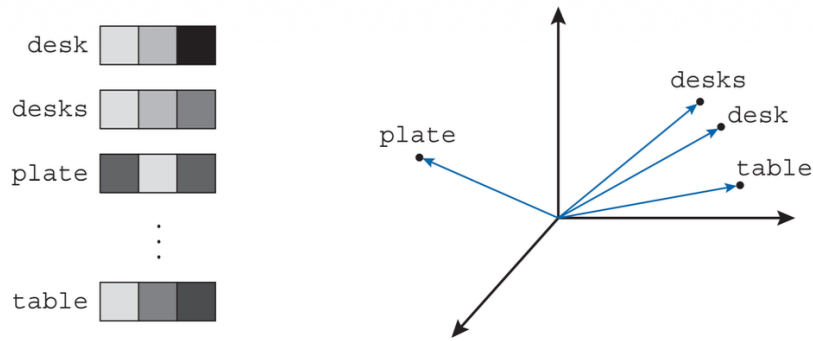


Abbildung 1: Ein 3-dimensionaler Vektorraum, in welchem Embeddings abgebildet werden. Wörter mit einer semantischen Ähnlichkeit liegen nah aneinander ([Camacho-Collados and Pilehvar 2020](#)).

[Jurafsky and Martin 2025](#)).

2.2.3 Verteilungsansatz

“You shall know a word by the company it keeps.”

[Firth \(1957\)](#)

Der sogenannte Verteilungsansatz, erstmals von [Firth \(1957\)](#) vorgestellt, liefert das fundamentale Prinzip für Embeddings. Die Kernaussage besteht darin, dass ein Wort dadurch charakterisiert wird, in welchem Kontext es sich befindet. Mit anderen Worten: Wörter, die in ähnlichen Kontexten auftreten, haben tendenziell eine ähnliche Bedeutung. Beispielsweise haben die Planeten Jupiter und Venus eine ähnliche semantische Bedeutung, weil sie auch in ähnlichen Zusammenhängen vorkommen, wie zum Beispiel im Sonnensystem, Sternen, Astronomie etc.. Die statistische Auswertung darüber, welche Wörter typischerweise zusammen auftreten, ermöglicht Rückschlüsse auf ihre semantische Beziehung. Dieses gemeinsame Auftreten von Wörtern wird als Ko-Okkurrenz bezeichnet ([Camacho-Collados and Pilehvar 2020](#)).

2.2.4 Ähnlichkeitsmaße im semantischen Raum

Um die Ähnlichkeit zwischen zwei Zielwörtern v und w zu berechnen, wird eine geeignete Metrik benötigt. In der Praxis hat sich hierfür insbesondere die *Kosinus-Ähnlichkeit* bewährt. Sie misst die Ähnlichkeit zweier Vektoren anhand ihres Winkels im semantischen Raum und eignet sich besonders zur Bewertung semantischer Beziehungen zwischen Wörtern, Sätzen oder Dokumenten. Die Kosinus-Ähnlichkeit zweier Vektoren v und w ist

definiert als:

$$\cos(\theta) = \frac{v \cdot w}{\|v\| \|w\|}. \quad (1)$$

Dabei bezeichnet $\|v\|$ die euklidische Norm des Vektors v . Die Metrik nimmt Werte im Intervall $[-1, 1]$ an, wobei ein Wert nahe 1 auf eine hohe Ähnlichkeit, ein Wert nahe 0 auf geringe Ähnlichkeit und ein Wert nahe -1 auf entgegengesetzte Richtungen der Vektoren hinweist. Aufgrund dieser Eigenschaften ist die Kosinus-Ähnlichkeit eine der am häufigsten verwendeten Metriken zur Berechnung von Wortähnlichkeiten in Vektorraummodellen ([Jurafsky and Martin 2025](#)).

2.3 Wort-Embeddings und zentrale Modelle

Wort-Embeddings sind ein zentraler Baustein moderner NLP-Systeme, weil sie Sprache in eine Form bringen, mit der Maschinen sinnvoll rechnen können. Statt Wörter nur als diskrete Symbole zu behandeln, werden sie als numerische Vektoren repräsentiert, die semantische Eigenschaften tragen. Dadurch lässt sich Bedeutung nicht nur speichern, sondern auch geometrisch vergleichen: Ähnliche Wörter liegen im Vektorraum näher beieinander als unähnliche. Grundsätzlich haben sich zwei große Modellfamilien etabliert, um solche Wortvektoren zu erzeugen. Die erste Familie konstruiert Embeddings direkt aus beobachteten Häufigkeiten im Text (zählbasierte Modelle). Die zweite Familie lernt Embeddings indirekt über eine Vorhersageaufgabe in neuronalen Netzen (vorhersagebasierte Modelle). Beide Ansätze folgen derselben Idee, setzen diese Intuition aber technisch unterschiedlich um ([Camacho-Collados and Pilehvar 2020](#)).

2.3.1 Ko-Okkurrenz-Matrix

In einer Ko-Okkurrenz-Matrix wird festgehalten, wie häufig Wörter gemeinsam in einem definierten Kontext auftreten. Sie wird insbesondere in der distributionellen Semantik verwendet, um statistische Zusammenhänge zwischen Wörtern eines Textkorpus zu modellieren. Ein Korpus sei dabei als endliche Sequenz von Tokens zu verstehen, aus der sich ein Vokabular V aller unterschiedlichen Wörter ableiten lässt ([Camacho-Collados and Pilehvar 2020](#)).

$$X = \begin{array}{c} \begin{array}{c} I \\ like \\ enjoy \\ deep \\ learning \\ NLP \\ flying \\ . \end{array} \begin{bmatrix} 0 & 2 & 1 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

Abbildung 2: Die Ko-Okkurrenzmatrix X der Texte “I like deep learning.”, “I like NLP.”, “I enjoy flying.” ([thumbjinx](#)).

Sei nun V das Vokabular des Korpus mit $|V|$ unterschiedlichen Wörtern. Die Ko-Okkurrenz-Matrix $C \in \mathbb{R}^{|V| \times |V|}$ ist eine quadratische Matrix, deren Eintrag $C_{i,j}$ die Anzahl der Beobachtungen angibt, in denen das Kontextwort $w_j \in V$ innerhalb eines festgelegten Kontextfensters des Zielwortes $w_i \in V$ im Korpus auftritt. Der Kontext eines Zielwortes wird typischerweise durch ein Fenster fester Größe definiert, das eine bestimmte Anzahl von Wörtern links und rechts des Zielwortes umfasst. Alternativ kann der Kontext auf satz-basierter Ebene bestimmt werden, indem alle Wörter desselben Satzes als kontextuell betrachtet werden. Ein hoher Wert von $C_{i,j}$ bedeutet somit, dass w_j häufig im definierten Kontext von w_i auftritt und weist auf eine hohe gemeinsame Auftretens-Frequenz beider Wörter im Korpus hin ([Turney and Pantel 2010](#); [Jurafsky and Martin 2025](#)).

2.3.2 Zählbasierte Modelle

Eines der ersten und zugleich einfachsten Verfahren sind zählbasierte Embeddings. Ihr Ausgangspunkt ist eine Ko-Okkurrenzmatrix, die festhält, wie oft Wörter gemeinsam im selben Kontext auftreten. Der Gedanke dahinter ist unmittelbar: Wörter, die in ähnlichen Umgebungen erscheinen, sollen auch ähnlich repräsentiert werden. Jede Zeile der Matrix kann daher als Vektor eines Wortes verstanden werden. In der Praxis werden solche Rohmatrizen meist weiterverarbeitet, etwa durch Gewichtungen oder Dimensionsreduktion, um kompaktere und aussagekräftigere Vektoren zu erhalten. Die bloße Anzahl von Wort-Zusammenhängen ist nicht ausreichend, um die Stärke der Assoziation zwischen zwei Wörtern zuverlässig zu bewerten. Dabei kristallisiert sich folgendes Problem heraus:

Häufig auftretende Wörter können hohe Ko-Okkurrenzwerte erzeugen, ohne tatsächlich semantisch stark miteinander verbunden zu sein. Daher wird ein normalisiertes Maß zur Bewertung von Wort-Assoziationen benötigt. Ein häufig verwendetes Maß ist die *Positive Pointwise Mutual Information* (PMI). PMI setzt die gemeinsame Auftretenswahrscheinlichkeit zweier Wörter ins Verhältnis zu den Wahrscheinlichkeiten ihrer unabhängigen Vorkommen. Dadurch wird gemessen, ob zwei Wörter häufiger gemeinsam auftreten, als es unter der Annahme statistischer Unabhängigkeit zu erwarten wäre. Die PMI zweier Wörter w_1 und w_2 ist definiert als

$$\text{PMI}(w_1, w_2) = \log_2 \frac{P(w_1, w_2)}{P(w_1) P(w_2)}. \quad (2)$$

Dabei bezeichnet $P(w)$ die geschätzte Wahrscheinlichkeit des Auftretens des Wortes w im Korpus, welche direkt aus seiner relativen Häufigkeit berechnet werden kann. $P(w_1, w_2)$ beschreibt entsprechend die geschätzte Wahrscheinlichkeit, dass die beiden Wörter w_1 und w_2 gemeinsam innerhalb eines definierten Kontextes im Korpus auftreten. Ein positiver PMI-Wert zeigt an, dass zwei Wörter häufiger gemeinsam auftreten, als es bei unabhängigem Auftreten zu erwarten wäre, während ein Wert von 0 auf statistische Unabhängigkeit hindeutet. Negative PMI-Werte weisen darauf hin, dass die Wörter seltener gemeinsam auftreten als erwartet. In der Praxis wird daher häufig die *Positive PMI* (PPMI) verwendet, bei der negative Werte auf 0 gesetzt werden ([Camacho-Collados and Pilehvar 2020](#); [Jurafsky and Martin 2025](#)).

2.3.3 Vorhersagebasierte Modelle

Im letzten Jahrzehnt wurden mit der Zunahme von Deep Learning die Embedding-Repräsentationen durch neuronale Netze so dominant, dass sie die konventionellen zählbasierten Modelle weitgehend verdrängt haben. Während zählbasierte Verfahren auf Häufigkeiten beruhen, entstehen vorhersagebasierte Embeddings durch Lernen: Ein neuronales Netz wird darauf trainiert, Wörter aus ihrem Kontext vorherzusagen oder umgekehrt. Die dabei entstehenden Gewichte bilden dichte, niedrig-dimensionale Vektoren - genau das, was im NLP als Embeddings genutzt wird. Diese Vektoren sind deutlich kompakter als One-Hot-Kodierungen und enthalten zugleich semantische Struktur, weshalb sie sich als Standardrepräsentation in vielen Anwendungen durchgesetzt haben. Im Gegensatz zu

klassischen, sehr hochdimensionalen und sparsamen Darstellungen sind Embeddings kurze, dichte Vektoren. Sie besitzen nicht eine Dimension pro Wort im gesamten Vokabular, sondern liegen typischerweise in einem Bereich von etwa 50 bis 1000 Dimensionen. „Dicht“ bedeutet dabei auch, dass die Einträge nicht nur 0 oder 1 annehmen, sondern reelle Werte enthalten, inklusive negativer Bereiche. Dadurch können feine Bedeutungsunterschiede abgebildet werden ([Russell and Norvig 2009](#); [Camacho-Collados and Pilehvar 2020](#)).

2.3.3.1 Word2vec Eine besonders verbreitete Methode zum Lernen solcher Wortvektoren ist "word2vec". Sie gilt als schnell und effizient im Training und erzeugt statische Embeddings, also einen festen Vektor pro Wort im Vokabular. Das unterscheidet word2vec von kontextuellen Verfahren, bei denen dasselbe Wort je nach Umgebung unterschiedliche Vektoren erhält (mehr dazu in Kapitel 3). Word2vec umfasst zwei Trainingsarchitekturen: Continuous Bag-of-Words (CBOW) und Skip-Gram. Beiden liegt die Annahme des Verteilungsansatzes zugrunde, sie unterscheiden sich jedoch darin, in welche Richtung die Vorhersage erfolgt. Beim *Continuous Bag-of-Words-Modell (CBOW)* wird ein Zielwort anhand der umgebenden Kontextwörter innerhalb eines definierten Fensters vorhergesagt, wobei die Reihenfolge der Kontextwörter nicht berücksichtigt wird ([Camacho-Collados and Pilehvar 2020](#); [Jurafsky and Martin 2025](#)).

2.3.3.2 Skip-Gram Im Gegensatz zu CBOW verfolgt das Skip-Gram-Modell die umgekehrte Zielsetzung: Ausgehend von einem Zielwort werden die Wörter seines Kontextes vorhergesagt. Für jedes Wort im Korpus werden somit mehrere Zielwort-Kontext-Paare erzeugt. In der Anwendung wird häufig die Variante mit *Negative Sampling* verwendet. Dabei wird das Training als binäres Klassifikationsproblem formuliert: Beobachtete Zielwort-Kontext-Paare gelten als positive Beispiele, während zufällig gezogene Wortpaare als negative Beispiele dienen. Ein logistischer Klassifikator lernt, zwischen beiden Fällen zu unterscheiden, indem er die Wahrscheinlichkeit realer Kontexte maximiert und die zufälliger Paare minimiert. Die dabei gelernten Gewichtsmatrizen stellen schließlich die Wort-Embeddings dar ([Jurafsky and Martin 2025](#)).

2.3.3.3 GloVe GloVe (Global Vectors) kombiniert Ideen zählbasierter und vorhergesagter Ansätze. Das Modell nutzt globale Ko-Okkurrenzstatistiken des gesamten Korpus und formuliert das Lernen als Optimierungsproblem. Ziel ist es, Wortvek-

toren so zu bestimmen, dass ihr Skalarprodukt den Logarithmus der beobachteten Ko-Okkurrenzhäufigkeit approximiert. Formal wird eine Kostenfunktion minimiert, die die Differenz zwischen dem Skalarprodukt zweier Wortvektoren und einem transformierten Ko-Okkurrenzwert gewichtet erfasst. Auf diese Weise entstehen dichte Vektorrepräsentationen, die globale statistische Informationen des Korpus in kompakter Form kodieren. Die zugrunde liegende Kostenfunktion ist gewichtet, um sehr seltene oder extrem häufige Ko-Okkurrenzen weniger stark zu berücksichtigen ([Jurafsky and Martin 2025](#)).

Aus wissenschaftlicher und praxisorientierter Sicht greift dieser statische Ansatz jedoch zu kurz. Da klassische Wort-Embeddings wie Word2Vec oder GloVe jedem Token unabhängig vom Kontext einen festen Vektor zuweisen, versagen sie systematisch bei Polysemie und Homonymie. Für ein Enterprise-System im Retail-Bereich bedeutet dies eine gravierende Einschränkung: Das Wort ‚Bank‘ (Sitzmöbel vs. Finanzinstitut) oder produktrelevante Begriffe wie ‚Schale‘ (Obstschale vs. Verpackungselement) können nicht differenziert werden. Die starre Vektorrepräsentation ignoriert die syntaktische Umgebung komplett, was in Folgesystemen zu Informationsverlust führt und eine manuelle, fehleranfällige Vorab-Kontextualisierung erzwingt.

3 Moderne Embeddings

3.1 Von statischen zu kontextualisierten Repräsentationen

In der Praxis des Natural Language Processing (NLP) reichen klassische statische Wort-Embeddings häufig nicht aus, um kontextabhängige Bedeutungsunterschiede von Wörtern adäquat zu erfassen. Während statische Verfahren jedem Wort genau einen festen Vektor zuordnen, erzeugen kontextualisierte Embeddings für jedes Auftreten eines Wortes eine eigene Repräsentation, die vom jeweiligen Satzkontext abhängt. Dadurch wird nicht nur die allgemeine Bedeutung eines Wortes repräsentiert, sondern auch dessen konkrete Verwendung innerhalb einer gegebenen Textsequenz. Ein zentrales Problem statischer Embeddings besteht darin, dass viele Wörter polysem sind, also mehrere Bedeutungen besitzen. Da statische Modelle diese Mehrdeutigkeit nicht berücksichtigen können, führen sie unterschiedliche Bedeutungen eines Wortes auf denselben Vektor zurück. Kontextualisierte Embeddings lösen dieses Problem, indem sie die Bedeutung eines Wortes relativ zu

seinem Kontext modellieren. In [Tabelle 1](#) wird die Wichtigkeit von Kontextbezug anhand mehrdeutiger Wörter illustriert.

Tabelle 1: Kontextabhängige Bedeutungen mehrdeutiger Wörter

Wort	Kontext 1	Kontext 2
<i>Maus</i>	Er bedient den Computer mit der Maus.	Eine Maus ist ein kleines Tier.
<i>Bank</i>	Sie hat Geld von der Bank abgehoben.	Sie saßen auf der Bank.

Obwohl es sich jeweils um dasselbe Wort handelt, ergeben sich unterschiedliche Bedeutungen, die erst durch den jeweiligen Kontext eindeutig werden. Kontextualisierte Embeddings sind in der Lage, diese Unterschiede abzubilden, indem sie für jedes Vorkommen eines Wortes eine kontextspezifische Repräsentation erzeugen. Bevor kontextualisierte Embeddings mehr im Detail bearbeitet werden, wird im folgenden Unterkapitel die fundamentale Transformer-Architektur behandelt ([Devlin et al. 2019](#); [Peters et al. 2018](#); [Jurafsky and Martin 2025](#)).

3.2 Transformer als Grundlage kontextualisierter Embeddings

Transformer bilden die technische Grundlage vieler moderner NLP-Modelle, die kontextualisierte Token- und Textrepräsentationen erzeugen. Ursprünglich wurden sie als Sequenz-zu-Sequenz-Modelle entwickelt, bei denen eine Eingabesequenz in eine Ausgabesequenz überführt wird. In bestimmten Transformer-Architekturen, insbesondere bei Decoder-basierten Modellen, erfolgt die Generierung autoregressiv. Das bedeutet, dass die Ausgabe schrittweise erzeugt wird und bereits generierte Tokens als zusätzlicher Kontext für die Vorhersage des nächsten Tokens genutzt werden ([Camacho-Collados and Pilehvar 2020](#); [Vaswani et al. 2023](#)).

Die ursprüngliche Transformer-Architektur folgt einer Encoder-Decoder-Struktur. Der Encoder verarbeitet die Eingabesequenz und erzeugt kontextualisierte Repräsentationen, während der Decoder diese Repräsentationen nutzt, um die Ausgabesequenz zu generieren. Daneben existieren auch Varianten, die ausschließlich aus einem Encoder, wie BERT in [Unterunterabschnitt 3.3.1](#), oder ausschließlich aus einem Decoder, wie GPT in [Unterunterabschnitt 3.3.2](#), bestehen. Im Gegensatz zu früheren Sequenzmodellen, etwa rekurrenten

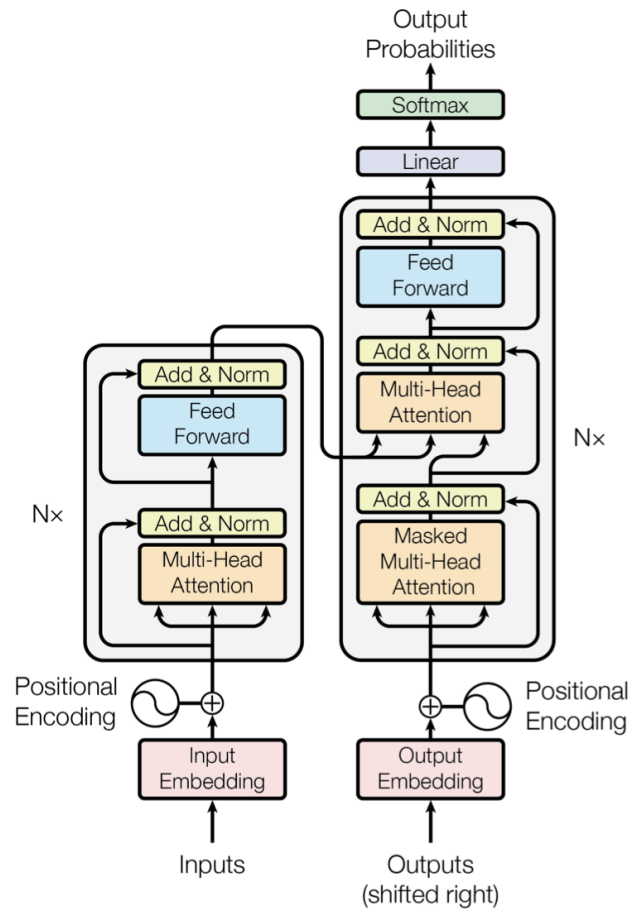


Abbildung 3: Transformer-Architektur nach [Camacho-Collados and Pilehvar \(2020\)](#)

neuronalen Netzen, verzichten Transformer auf rekurrente Strukturen. Stattdessen basieren sie auf Self-Attention-Mechanismen und Feed-Forward-Schichten. Dadurch können sie effizient parallelisiert werden und weitreichende Abhängigkeiten innerhalb von Sequenzen modellieren. Der sogenannte Self-Attention-Mechanismus ermöglicht es dem Modell, Tokens innerhalb einer Sequenz verschieden stark zu gewichten und dadurch syntaktische sowie semantische Beziehungen unabhängig von ihrer Distanz zu erfassen. Dies ist grundlegend für die Erzeugung kontextualisierter Embeddings ([Camacho-Collados and Pilehvar 2020](#); [Vaswani et al. 2023](#)).

Die grundlegende Architektur eines Transformers ist in [Abbildung 3](#) dargestellt. Sie besteht aus zwei Hauptkomponenten: einem Encoder auf der linken Seite und einem Decoder auf der rechten Seite, die jeweils aus mehreren identischen Schichten aufgebaut sind. Der Encoder kombiniert zunächst die Eingabe-Embeddings mit Positionsinformationen, sogenannten Positional Encodings, um die Reihenfolge der Tokens zu berücksichtigen.

tigen. Anschließend durchläuft die Sequenz mehrere Encoder-Blöcke, die aus Multi-Head Self-Attention und einer Feed-Forward-Schicht bestehen. Residualverbindungen und Normalisierungsschritte, in der Abbildung als „Add & Norm“ dargestellt, stabilisieren dabei das Training. Der Decoder erhält als Eingabe die bisher generierte Ausgabesequenz. Er verwendet ebenfalls Self-Attention- und Feed-Forward-Schichten, enthält jedoch zusätzlich eine maskierte Self-Attention. Diese stellt sicher, dass bei der Vorhersage eines Tokens nur vorherige Tokens berücksichtigt werden. Darüber hinaus nutzt der Decoder eine Attention-Schicht, die auf die Repräsentationen des Encoders zugreift, um relevante Informationen aus der Eingabesequenz einzubeziehen. Durch das mehrfache Stapeln dieser Blöcke, in der Abbildung mit $N \times$ gekennzeichnet, entsteht ein tiefes Modell, das komplexe Abhängigkeiten innerhalb von Sequenzen erfassen kann (Vaswani et al. 2023; Camacho-Collados and Pilehvar 2020).

3.2.1 Self-Attention

Self-Attention, auch als *Intra-Attention* bezeichnet, ist ein Mechanismus, durch den ein Modell beim Verarbeiten eines Tokens andere Tokens derselben Sequenz berücksichtigen kann. Dadurch wird die Repräsentation eines Wortes dynamisch an seinen jeweiligen Kontext angepasst. Der Mechanismus greift damit die Grenze statischer Embeddings auf, die in [Unterabschnitt 3.1](#) beschrieben wurde: Ein Wort erhält nicht mehr eine feste Repräsentation, sondern wird abhängig von seinem Auftreten im Satz interpretiert. Im Gegensatz zur klassischen Attention, bei der Beziehungen zwischen unterschiedlichen Sequenzen, beispielsweise zwischen Encoder- und Decoder-Sequenz, modelliert werden, betrachtet Self-Attention ausschließlich Abhängigkeiten innerhalb derselben Sequenz. Formal basiert der Mechanismus auf drei Repräsentationen eines Tokens: Query (Q), Key (K) und Value (V). Für jedes Token wird berechnet, wie stark seine Query-Repräsentation mit den Key-Repräsentationen aller Tokens der Sequenz übereinstimmt. Die daraus entstehenden Gewichtungen werden anschließend verwendet, um eine gewichtete Kombination der Value-Vektoren zu bilden. In kompakter Form lässt sich der Mechanismus wie folgt darstellen:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (3)$$

Dabei bezeichnet d_k die Dimension der Key-Vektoren und dient der Skalierung der Skalarprodukte. Die Formel verdeutlicht, dass die Bedeutung eines Tokens nicht isoliert entsteht, sondern durch die gewichtete Einbeziehung seines Kontexts bestimmt wird ([Vaswani et al. 2023](#); [Camacho-Collados and Pilehvar 2020](#)).

3.3 Kontextualisierte Embeddings

Anders als statische Wort-Embeddings repräsentieren kontextualisierte Embeddings Wörter stets in einem gegebenen Kontext. Während statische Verfahren jedem Wort genau einen festen Vektor zuordnen, erzeugen kontextualisierte Modelle für jedes Auftreten eines Wortes eine eigene Repräsentation, die vom jeweiligen Satzkontext abhängt. Dadurch können nicht nur semantische, sondern auch syntaktische Eigenschaften eines Wortes im Satz erfasst werden. Ein weiterer wichtiger Unterschied besteht darin, dass der Input für solche Modelle nicht nur ein einzelnes Wort ist, sondern eine gesamte Textsequenz. Erst dadurch kann das Modell die Umgebung eines Tokens berücksichtigen und dessen Bedeutung kontextabhängig repräsentieren. Dies ermöglicht es, Embeddings nicht nur für einzelne Wörter zu nutzen, sondern auch für komplexere linguistische Aufgaben, bei denen die Beziehung zwischen Wörtern im Kontext eine zentrale Rolle spielt. Formal lässt sich dies wie folgt darstellen: Gegeben sei eine Eingabesequenz von Token $x_1, \dots, x_n$. Ein kontextualisiertes Modell erzeugt für jedes Token x_i einen Ausgabevektor h_i^L aus der letzten Schicht L . Dieser Vektor kann als Repräsentation der Bedeutung von x_i im Kontext des gesamten Satzes $x_1, \dots, x_n$ interpretiert werden. In der Praxis werden häufig nicht nur die Ausgaben der letzten Schicht verwendet, sondern mehrere Schichten kombiniert, beispielsweise durch Mittelung der letzten vier Layer $(h_i^L, h_i^{L-1}, h_i^{L-2}, h_i^{L-3})$, um stabilere Repräsentationen zu erhalten ([Camacho-Collados and Pilehvar 2020](#)).

3.3.1 BERT

Mit der Einführung von Transformer-Learning-Modellen und ihrem Potenzial beim Kodieren von Textsequenzen wurde BERT entwickelt. BERT steht für „Bidirectional Encoder Representations from Transformers“ und basiert auf einer Encoder-Architektur des Transformers. Ein zentrales Merkmal von BERT ist die bidirektionale Kontextverarbeitung: Das Modell berücksichtigt bei der Repräsentation eines Wortes sowohl den linken als auch den rechten Kontext gleichzeitig. Um dies zu ermöglichen, verwendet BERT das sogenannte

Masked Language Modeling. Dabei werden einzelne Tokens in der Eingabe zufällig maskiert, und das Modell lernt, diese anhand des verbleibenden Kontexts zu rekonstruieren. Beispielsweise wird der Satz

“Die Struktur der Zell Membran ist bekannt als flüssiges Mosaik”

zu

“Die Struktur der Zell [MASK] ist bekannt [MASK] flüssiges Mosaik”

transformiert. Das Modell versucht anschließend, die maskierten Tokens korrekt vorherzusagen. Dadurch kann es Informationen aus beiden Richtungen des Kontexts nutzen, was einen entscheidenden Vorteil gegenüber unidirektionalen Modellen darstellt. Die resultierenden Embeddings sind kontextabhängig, sodass ein Wort je nach Verwendung im Satz unterschiedliche Vektorrepräsentationen erhält. Das ursprüngliche BERT-Modell basiert auf einer Word-Piece-Tokenisierung mit etwa 30 000 Subwords, einer maximalen Sequenzlänge von 512 Tokens und einer Modelltiefe von 12 Transformer-Schichten mit jeweils mehreren Attention-Heads. Insgesamt umfasst das Modell etwa 100 Millionen Parameter. In späteren Arbeiten wurde BERT weiterentwickelt und optimiert, woraus Modelle wie RoBERTa, ALBERT oder DistilBERT hervorgegangen sind ([Devlin et al. 2019](#); [Camacho-Collados and Pilehvar 2020](#)).

Kritisch zu betrachten ist bei Transformer-basierten Modellen wie BERT jedoch der massive Trade-off zwischen semantischer Präzision und rechnerischer Effizienz. Während die bidirektionale Aufmerksamkeitsmatrix (Self-Attention) eine exzellente Kontextualisierung erlaubt, skaliert der Rechenaufwand quadratisch mit der Sequenzlänge $O(N^2)$.

Im operativen Betrieb eines Enterprise Systems führt dies zu signifikanten Latenzproblemen bei Echtzeit-Anfragen. Zudem zeigt die Forschung (vgl. [Reimers and Gurevych \(2019\)](#)), dass die standardmäßigen Output-Vektoren der CLS-Token von BERT ohne feingranulares Domain-Finetuning oft eine kollabierte Anisotropie aufweisen, d.h. sie driften in einem engen Vektorraum zusammen und sind für direkte Kosinus-Ähnlichkeitsberechnungen paradoxerweise schlechter geeignet als einfachere, gemittelte statische Embeddings ([Reimers and Gurevych 2019](#)).

3.3.2 GPT

GPT basiert ebenfalls auf Transformer-Architekturen, verwendet jedoch ausschließlich den Decoder-Teil des Transformers und arbeitet im Gegensatz zu BERT autoregressiv. Dadurch eignet es sich insbesondere für generative Aufgaben wie Textfortsetzung oder Textgenerierung. Da GPT nur auf den linken Kontext zugreift, ist es ein unidirektionales Modell. Die Vorhersage erfolgt sequenziell von links nach rechts, wobei jedes neu erzeugte Token als zusätzlicher Kontext für die nächsten Vorhersagen dient. Diese Eigenschaft macht GPT besonders geeignet für Anwendungen im Bereich der Sprachgenerierung. Transformer-Modelle wie GPT haben frühere rekurrente Ansätze in vielen Aufgaben übertroffen, da sie lange Abhängigkeiten innerhalb von Texten effizient modellieren können, ohne auf rekursive Strukturen angewiesen zu sein. Auch hier ermöglicht der Attention-Mechanismus die Berücksichtigung relevanter Kontextinformationen. Auf dieser Modellfamilie bauen auch moderne dialogorientierte KI-Systeme wie ChatGPT auf, das von OpenAI entwickelt wurde und GPT-Modelle für interaktive Sprachverarbeitung nutzt. GPT stellt damit eine zentrale technische Grundlage heutiger generativer KI-Anwendungen dar, insbesondere für Chatbots, Assistenzsysteme und textbasierte Enterprise-Anwendungen ([Radford et al. 2019](#); [Camacho-Collados and Pilehvar 2020](#)).

3.3.3 Ähnlichkeitsmaße für kontextualisierte Embeddings

Bei statischen Embeddings wird die Ähnlichkeit zwischen Wörtern häufig über geometrische Maße wie die Kosinus-Ähnlichkeit bestimmt, wie bereits in [Unterunterabschnitt 2.2.4](#) beschrieben. Dieses Prinzip lässt sich grundsätzlich auch auf kontextualisierte Embeddings übertragen, ist jedoch nicht immer risikofrei.

Ein zentrales Problem ist die sogenannte Anisotropie. Sie beschreibt die Eigenschaft, dass Embedding-Vektoren nicht gleichmäßig über den Vektorraum verteilt sind, sondern sich in bestimmten Richtungen konzentrieren. Dadurch können auch zufällig gewählte oder semantisch wenig verwandte Wörter vergleichsweise hohe Kosinus-Ähnlichkeiten aufweisen. [Ethayarajh \(2019\)](#) beschreibt Anisotropie entsprechend als die erwartete Kosinus-Ähnlichkeit zwischen zufällig gewählten Wortpaaren in einem Korpus. In einem ideal isotropen Vektorraum wären die Repräsentationen hingegen gleichmäßiger in alle Richtungen verteilt, wodurch Ähnlichkeitswerte aussagekräftiger interpretierbar wären. Um die Aus-

sagekraft geometrischer Ähnlichkeitsmaße bei anisotropen Vektorräumen zu verbessern, können Embeddings nachverarbeitet werden. Dazu zählen insbesondere die Zentrierung der Vektoren, etwa durch Abzug eines Mittelwertvektors, sowie die Normalisierung der Repräsentationen. Ziel solcher Schritte ist es, dominante gemeinsame Richtungen im Vektorraum abzuschwächen und semantische Unterschiede zwischen den Repräsentationen deutlicher hervortreten zu lassen ([Jurafsky and Martin 2025](#); [Ethayarajh 2019](#)).

3.4 Satz- und Dokument-Embeddings

Bisher wurden Embeddings hauptsächlich auf Wort- und Token-Ebene betrachtet. Für viele praktische Anwendungen im Enterprise-Kontext reicht diese Ebene jedoch nicht aus. Semantische Suche, Retrieval-Augmented Generation und Produktsuche arbeiten typischerweise nicht mit einzelnen Wörtern, sondern mit ganzen Suchanfragen, Produktbeschreibungen, Dokumentsegmenten oder Wissensbeiträgen. Daher werden Repräsentationen benötigt, die größere sprachliche Einheiten in einen gemeinsamen semantischen Vektorraum überführen. Solche Repräsentationen werden als Satz- oder Dokument-Embeddings bezeichnet ([Camacho-Collados and Pilehvar 2020](#); [Jurafsky and Martin 2025](#)).

Von klassischen Wort-Embedding-Verfahren wie CBOW oder Skip-Gram unterscheiden sich Satz- und Dokument-Embeddings dadurch, dass nicht einzelne Wörter, sondern vollständige Textsegmente als eine zusammenhängende Bedeutungseinheit repräsentiert werden. Während Wort-Embeddings semantische Beziehungen auf Wortebene abbilden, zielen Satz- und Dokument-Embeddings darauf ab, die Bedeutung größerer sprachlicher Einheiten vergleichbar zu machen.

Klassische Ansätze zur Repräsentation ganzer Texte sind Bag-of-Words, TF-IDF oder Latent Dirichlet Allocation (LDA). Beim Bag-of-Words-Modell wird ein Text durch die Häufigkeit seiner enthaltenen Wörter beschrieben. Dieses Verfahren ist einfach und effizient, berücksichtigt jedoch weder Wortreihenfolge noch Kontextinformationen. LDA geht darüber hinaus und modelliert Dokumente als Mischungen latenter Themen, die wiederum durch Wortverteilungen charakterisiert sind. Beide Verfahren können nützliche statistische Informationen über Dokumente liefern, erfassen semantische Ähnlichkeit jedoch nur eingeschränkt, da sie primär auf beobachteten Wortverteilungen beruhen ([Blei et al. 2003](#); [Jurafsky and Martin 2025](#); [Turney and Pantel 2010](#)).

Moderne Satz- und Dokument-Embeddings verfolgen einen anderen Ansatz. Sie erzeugen dichte Vektorrepräsentationen, die die Bedeutung einer gesamten Texteingabe abbilden sollen. Eine einfache Möglichkeit besteht darin, Token-Embeddings eines Transformer-Modells zu aggregieren, beispielsweise durch Mittelung der Token-Repräsentationen oder durch Verwendung eines speziellen Klassifikationstokens. Diese naive Aggregation ist jedoch nicht automatisch optimal für semantische Ähnlichkeitssuche. Insbesondere sind die Ausgaben allgemeiner BERT-Modelle nicht notwendigerweise dafür trainiert, dass semantisch ähnliche Sätze im Vektorraum nahe beieinanderliegen. BERT wurde ursprünglich für Aufgaben wie Masked Language Modeling und Next Sentence Prediction trainiert, nicht spezifisch für effizientes Retrieval oder Satzähnlichkeit ([Devlin et al. 2019](#); [Camacho-Collados and Pilehvar 2020](#)).

3.4.1 SBERT

Eine zentrale Weiterentwicklung stellt Sentence-BERT (SBERT) dar. SBERT basiert auf BERT- bzw. RoBERTa-Modellen, verwendet jedoch Siamese- oder Triplet- Netzwerkstrukturen, um semantisch aussagekräftige Satz-Embeddings zu erzeugen. Bei Siamese- oder Triplet-Netzwerkstrukturen werden zwei beziehungsweise drei Eingaben gemeinsam verarbeitet, um das Modell gezielt darauf zu trainieren, semantisch ähnliche Texte mit ähnlichen Vektorrepräsentationen und semantisch unähnliche Texte mit größeren Abständen im Vektorraum abzubilden. Dabei werden Sätze unabhängig voneinander kodiert und anschließend über die Kosinus-Ähnlichkeit verglichen. Dadurch eignet sich SBERT deutlich besser für Aufgaben wie semantische Suche, Clustering oder Duplikaterkennung, da Embeddings vorab berechnet und effizient in einem Vektorindex gespeichert werden können ([Reimers and Gurevych 2019](#)).

Obwohl Architekturen wie SBERT das Problem der Recheneffizienz durch siamesische Netzwerke für Paarbeurteilungen reduzieren, bleibt die Aggregation längerer Dokumente zu einem einzigen Vektor problematisch. Die Komprimierung eines längeren, mehrthematischen Fließtextes in eine fixe Vektorrepräsentation kann dazu führen, dass spezifische, lokal isolierte Details, beispielsweise innerhalb einer Produktbeschreibung, im Gesamtvektor nur unzureichend abgebildet werden. Für die Architektur eines Enterprise-Wissensmanagements bedeutet dies, dass Dokument-Embeddings selten als monolithische Lösung betrachtet werden sollten, sondern häufig durch geeignete Chunking-Strategien so-

wie hybride Suchkomponenten, etwa die Kombination aus lexikalischer und semantischer Suche, ergänzt werden müssen ([Reimers and Gurevych 2019](#); [Thakur et al. 2021](#)).

4 Gemeinsame Grundarchitektur Embedding-basierter Enterprise-Systeme

Die in den vorherigen Kapiteln dargestellten Embedding-Methoden und theoretischen Grundlagen bilden die Basis für verschiedene Anwendungen im Enterprise-Kontext. Obwohl sich Use-Cases wie semantische Suche, Retrieval-Augmented Generation oder Empfehlungssysteme in ihrer Zielsetzung unterscheiden, folgen sie häufig einem ähnlichen technischen Prinzip. Dieses besteht darin, relevante Unternehmensdaten aufzubereiten, in semantische Repräsentationen zu überführen, effizient zu speichern und anschließend für anwendungsspezifische Vergleiche nutzbar zu machen. Unter der gemeinsamen Grundarchitektur wird in dieser Arbeit daher ein abstrahiertes Modell verstanden, das die zentralen Verarbeitungsschritte solcher Systeme beschreibt. Es ist nicht auf einen einzelnen Anwendungsfall beschränkt, sondern kann je nach Zielsetzung angepasst werden. Im Folgenden werden zunächst die gemeinsamen Basisschritte beschrieben: Datenaufbereitung, Embedding-Erzeugung und Speicherung im Vektorindex. Anschließend wird gezeigt, wie diese Grundlage in zentralen Anwendungsmustern genutzt wird, insbesondere im Embedding-Retrieval, in Retrieval-Augmented Generation sowie in Embedding-basierten Empfehlungssystemen. Somit bildet dieses Kapitel das Fundament für die in [Abschnitt 5](#) gezeigten Prototypen.

4.1 Datenaufbereitung

Der erste Schritt der gemeinsamen Architektur besteht in der Aufbereitung der zugrunde liegenden Daten. Im Unternehmenskontext liegen relevante Informationen in sehr unterschiedlichen Formen vor: von unstrukturierten Texten wie Dokumenten, E-Mails oder Chatverläufen bis hin zu strukturierten Daten aus Datenbanken. Damit diese Informationen für Embedding-Modelle nutzbar werden, müssen sie zunächst bereinigt, vereinheitlicht und in eine geeignete Repräsentationsform überführt werden. Zur Datenaufbereitung zählen beispielsweise die Entfernung irrelevanter Inhalte, die Normalisierung von Text, die Vereinheitlichung unterschiedlicher Formate sowie die Auswahl relevanter Metadaten. Bei

längeren Dokumenten ist zusätzlich eine Segmentierung erforderlich. Beim sogenannten Chunking werden größere Texte in kleinere, semantisch sinnvolle Einheiten unterteilt. Dies ist besonders relevant, da in vielen Anwendungen nicht ein vollständiges Dokument, sondern ein bestimmter Abschnitt für eine Anfrage oder Weiterverarbeitung ausschlaggebend ist. Die Qualität der Vorverarbeitung hat einen direkten Einfluss auf die später erzeugten Embeddings und damit auf die Leistungsfähigkeit nachgelagerter Anwendungen wie Retrieval, RAG oder Empfehlungssysteme. Werden Dokumente zu grob segmentiert, können relevante Details im Gesamtvektor untergehen. Werden sie hingegen zu kleinteilig aufgeteilt, kann wichtiger Kontext verloren gehen. Die Aufbereitung der Daten ist daher kein rein vorbereitender Schritt, sondern ein zentraler Bestandteil der Gesamtarchitektur. ([Kamath et al. 2024](#))

4.2 Embedding-Erzeugung und Speicherung

Nach der Aufbereitung werden die Daten mithilfe eines geeigneten Embedding-Modells in Vektorrepräsentationen überführt. Grundlage dafür ist das in [Abschnitt 2](#) eingeführte Vektorraum-Modell: Texte, Dokumentsegmente, Produktinformationen, Nutzerprofile oder Suchanfragen werden als Punkte in einem gemeinsamen Raum dargestellt, in dem inhaltlich ähnliche Objekte nahe beieinanderliegen. Dadurch werden sprachliche oder strukturierte Informationen in eine maschinenlesbare Form übertragen, die rechnerisch verglichen und weiterverarbeitet werden kann.

Die konkrete Methodik hängt vom jeweiligen Anwendungsfall ab. Für semantische Suche und RAG werden vor allem Satz- und Dokument-Embeddings benötigt, da hier vollständige Anfragen, Textabschnitte oder Wissensbeiträge miteinander verglichen werden. Für Empfehlungssysteme können dagegen neben textuellen Produktrepräsentationen auch Nutzer-, Produkt- oder Interaktionsdaten eingebettet werden. Wie in den vorherigen Kapiteln beschrieben, kommen hierfür insbesondere Transformer-basierte Modelle infrage, da sie kontextabhängige Repräsentationen erzeugen können ([Camacho-Collados and Pilehvar 2020](#)).

Wie in den folgenden Abschnitten näher erläutert wird, können erzeugten Embeddings entweder im Vorhinein berechnet und persistent gespeichert oder dynamisch zur Laufzeit erzeugt werden. Die vorberechneten Vektoren werden typischerweise in einem Vektorindex

gespeichert. Dieser speichert die Embeddings gemeinsam mit Verweisen auf die ursprünglichen Datenobjekte und ermöglicht später den effizienten Vergleich einer neuen Anfrage mit den bereits eingebetteten Beständen. Damit stellt der Vektorindex das Bindeglied zwischen der einmaligen Einbettung bestehender Datenbestände und der späteren Nutzung in konkreten Anwendungsszenarien dar. ([Karpukhin et al. 2020](#))

4.3 Embedding-Retrieval

Embedding-Retrieval bezeichnet die Nutzung von Vektorrepräsentationen zur Suche nach inhaltlich relevanten Objekten. Eine Nutzeranfrage (Query) und potenzielle Treffer, etwa Dokumente, Produktbeschreibungen oder Wissenseinträge, werden in denselben Vektorraum eingebettet und anschließend anhand eines Ähnlichkeitsmaßes verglichen. Dadurch können auch Inhalte gefunden werden, die nicht dieselben Begriffe wie die Anfrage verwenden, aber semantisch verwandt sind. Die Relevanz eines Treffers ergibt sich aus der Nähe zwischen Query-Embedding und Dokument-Embedding, beispielsweise auf Basis der Kosinus-Ähnlichkeit (siehe [Gleichung 1](#) und [Unterunterabschnitt 3.3.3](#)). Dieses Prinzip bildet die Grundlage für skalierbare semantische Suche und für Retrieval-Komponenten in RAG-Systemen. ([Karpukhin et al. 2020](#); [Reimers and Gurevych 2019](#))

4.3.1 Bi-Encoder und Cross-Encoder

Um die Funktionsweise von Embedding-Retrieval architektonisch einzuordnen, ist die Unterscheidung zwischen Bi-Encodern und Cross-Encodern relevant. Beide Ansätze beschreiben, wie eine Suchanfrage und ein potenziell relevantes Dokument verarbeitet und hinsichtlich ihrer Relevanz bewertet werden, unterscheiden sich jedoch deutlich in Effizienz, Speicherbarkeit der Embeddings und Genauigkeit der Bewertung.

Bei einem Bi-Encoder werden Query und Dokument getrennt voneinander kodiert. Beide Eingaben werden durch dasselbe oder durch zwei aufeinander abgestimmte Encoder-Modelle verarbeitet und in denselben Vektorraum projiziert. Dadurch entstehen zwei unabhängige Repräsentationen, die anschließend über ein Ähnlichkeitsmaß verglichen werden können. Der Relevanzscore entsteht daher nicht direkt im Modell, sondern durch den nachgelagerten Vergleich der beiden erzeugten Vektoren. Der zentrale Vorteil dieses Ansatzes besteht darin, dass Dokument-Embeddings unabhängig von konkreten Nutzeranfragen vorberechnet und in einem Vektorindex gespeichert werden können. Zur Laufzeit

muss lediglich die Query eingebettet und mit den gespeicherten Vektoren verglichen werden. Dadurch eignen sich Bi-Encoder besonders für semantische Suchsysteme und RAG-Anwendungen, bei denen große Dokumentbestände effizient durchsucht werden müssen (Karpukhin et al. (2020); Reimers and Gurevych (2019)).

Eine alternative Architektur stellen Cross-Encoder dar. Bei einem Cross-Encoder werden Query und Dokument nicht separat, sondern gemeinsam als Eingabe in ein Transformer-Modell gegeben. Dadurch kann das Modell direkte Interaktionen zwischen den Tokens der Anfrage und des Dokuments berücksichtigen und häufig präzisere Relevanzbewertungen erzeugen. Ein Cross-Encoder erzeugt dadurch keine unabhängig speicherbaren Dokument-Embeddings, sondern berechnet für jedes Query-Dokument-Paar direkt einen Relevanzscore. Der Nachteil besteht jedoch darin, dass für jedes dieser Paare eine separate Modellinferenz notwendig ist. Dokument-Embeddings können daher nicht in gleicher Weise vorbereitet und unabhängig von der Query gespeichert werden. Für die initiale Suche in großen Datenbeständen sind Cross-Encoder deshalb meist zu rechenintensiv, eignen sich jedoch gut für ein nachgelagertes Re-Ranking einer kleineren Menge bereits gefundener Kandidaten (Nogueira and Cho 2020).

In praktischen Retrieval-Systemen werden Bi-Encoder und Cross-Encoder daher häufig kombiniert. Zunächst ruft ein Bi-Encoder über den Vektorindex effizient eine Menge potenziell relevanter Kandidaten ab. Anschließend kann ein Cross-Encoder diese Kandidaten genauer bewerten und neu sortieren. Dadurch lassen sich Effizienz und Genauigkeit miteinander verbinden: Bi-Encoder ermöglichen eine skalierbare Vorauswahl, während Cross-Encoder eine präzisere Bewertung einer kleineren Kandidatenmenge erlauben (Nogueira and Cho 2020; Karpukhin et al. 2020).

4.4 Retrieval-Augmented Generation und generative Sprachmodelle

Retrieval-Augmented Generation (RAG) erweitert Embedding-Retrieval um eine generative Komponente. Während semantische Suche relevante Dokumente oder Dokumentsegmente direkt als Treffer zurückgibt, nutzt RAG die abgerufenen Inhalte als zusätzlichen Kontext für ein generatives Sprachmodell. Ziel ist es, Antworten zu erzeugen, die nicht ausschließlich auf dem im Modell gespeicherten Trainingswissen beruhen, sondern zusätz-

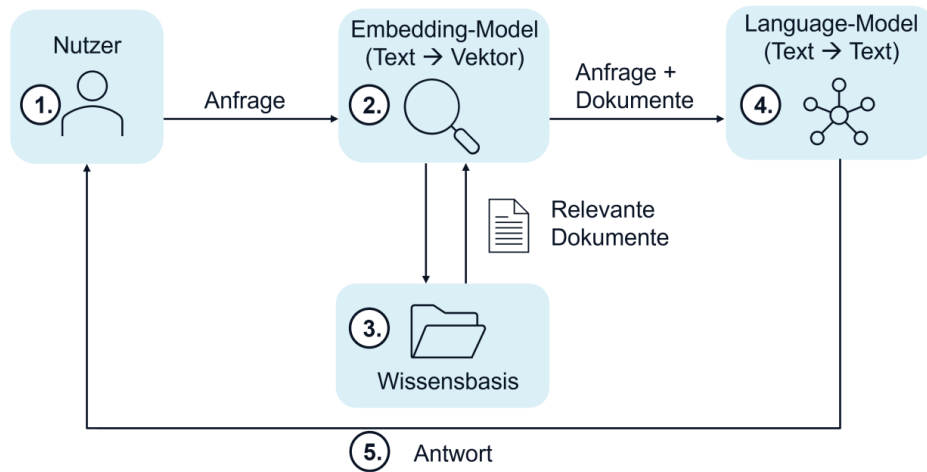


Abbildung 4: Schrittweise Darstellung von Retrieval-Augmented Generation ([Dünschede](#)).

lich auf externen, zur Laufzeit abgerufenen Informationen basieren. Dadurch wird eine Form des *Groundings* umgesetzt, bei der generierte Antworten auf konkret abrufbaren Unternehmensdaten beruhen. Die zuvor beschriebene Retrieval-Komponente bildet dabei die Grundlage: Relevante Dokumentensegmente werden nicht nur angezeigt, sondern gemeinsam mit der ursprünglichen Anfrage an ein Large Language Model (LLM) übergeben, das daraus eine strukturierte, natürlichsprachliche Antwort erzeugt ([Jurafsky and Martin 2025](#); [Kamath et al. 2024](#)).

Generative Sprachmodelle erzeugen Text sequenziell, indem sie auf Basis des bisherigen Kontexts die Wahrscheinlichkeit des nächsten Tokens modellieren. Formal lässt sich dies als bedingte Wahrscheinlichkeitsverteilung $P(w_i | w_{<i})$ beschreiben. Für das Verständnis der RAG-Architektur ist dabei weniger die interne Modellarchitektur entscheidend als die Fähigkeit des Sprachmodells, bereitgestellte Informationen kohärent in die Antwortgenerierung einzubinden ([Kamath et al. 2024](#)).

Der Prozess hinter einer RAG-basierten Anfrage lässt sich wie in [Abbildung 4](#) in folgende zentrale Schritte gliedern: Der Ablauf beginnt mit der Anfrage eines Nutzers (1). Diese wird durch ein Embedding-Modell in eine Vektorrepräsentation überführt (2) und mit den in der Wissensbasis gespeicherten Dokumenten verglichen (3). Die semantisch relevantesten Dokumente werden anschließend gemeinsam mit der ursprünglichen Anfrage an das Sprachmodell übergeben (4). Auf Grundlage dieses erweiterten Kontexts generiert das Modell schließlich eine Antwort für den Nutzer (5).

Während Embedding-Retrieval relevante Inhalte identifiziert und rankt, erweitert RAG

diesen Prozess um die sprachliche Aufbereitung der abgerufenen Informationen. Beide Anwendungsmuster beruhen jedoch auf derselben Grundarchitektur: Daten werden vorbereitet, eingebettet, in einem Vektorindex gespeichert und zur Laufzeit über semantische Ähnlichkeit abgerufen. Der Unterschied liegt in der Weiterverarbeitung der Treffer: Semantische Suche gibt die Ergebnisse direkt aus, während RAG sie als Kontext für ein Sprachmodell nutzt.

4.5 Embedding-Matching und Ranking

Das Ziel von Embedding-Matching und Ranking ist es, Beziehungen zwischen Objekten zu modellieren und daraus Relevanz- oder Ähnlichkeitsbewertungen abzuleiten. Während Embedding-Retrieval vor allem der Suche nach relevanten Dokumenten oder Wissenseinträgen dient, wird Embedding-Matching genutzt, um Objekte wie Nutzer, Produkte oder Interaktionen miteinander in Beziehung zu setzen. Dabei können beispielsweise Nutzerprofile, Produkte oder Interaktionsmuster als Vektoren in einem gemeinsamen Repräsentationsraum dargestellt werden. Die resultierenden Embeddings ermöglichen es, die Kompatibilität zwischen Objekten durch Ähnlichkeitsmaße zu quantifizieren. Ein zentraler Ansatz in diesem Bereich ist das Collaborative Filtering, bei dem Interaktionen zwischen Nutzern und Items genutzt werden, um latente Präferenzstrukturen zu lernen (Wang et al. 2025).

4.5.1 Collaborative Filtering

Collaborative Filtering zählt zu den essenziellen Verfahren in Empfehlungssystemen. Dabei werden auf Basis beobachteter Interaktionen zwischen Nutzern und Items personalisierte Empfehlungen generiert. Ausgangspunkt ist typischerweise eine User-Item-Matrix R , deren Eintrag $r_{u,i}$ die Interaktion eines Nutzers u mit einem Item i beschreibt. Solche Interaktionen können beispielsweise Käufe, Warenkorb-Einträge, Ansichten oder Bewertungen sein. Die Grundannahme besteht darin, dass aus ähnlichen Interaktionsmustern Rückschlüsse auf zukünftige Präferenzen gezogen werden können.

Ein besonders praxisrelevanter Ansatz ist item-basiertes Collaborative Filtering. Dabei werden nicht primär Nutzer miteinander verglichen, sondern Ähnlichkeiten zwischen Items berechnet. Zwei Items gelten als ähnlich, wenn sie häufig von denselben Nutzern konsumiert, bewertet oder anderweitig interagiert wurden. Für einen gegebenen Nutzer können

anschließend jene Produkte empfohlen werden, die den bereits konsumierten oder positiv bewerteten Produkten besonders ähnlich sind. Item-basierte Verfahren sind insbesondere deshalb attraktiv, weil Item-Item-Ähnlichkeiten vorab berechnet und zur Laufzeit effizient für Empfehlungen genutzt werden können. Formal lässt sich dabei die vorhergesagte Relevanz eines Zielitems i für einen Nutzer u als gewichtete Kombination der bereits bekannten Interaktionen des Nutzers darstellen:

$$\hat{r}_{u,i} = \frac{\sum_{j \in I_u} \text{sim}(i, j) \cdot r_{u,j}}{\sum_{j \in I_u} |\text{sim}(i, j)|} \quad (4)$$

Dabei bezeichnet I_u die Menge der Items, mit denen Nutzer u bereits interagiert hat. $\text{sim}(i, j)$ beschreibt die Ähnlichkeit zwischen dem Zielitem i und einem bereits bekannten Item j . Der Wert $r_{u,j}$ steht für die Stärke der Interaktion des Nutzers mit Item j , beispielsweise eine Bewertung oder ein gewichtetes Interaktionssignal. Ein Item erhält somit dann einen hohen vorhergesagten Score, wenn es den bisherigen Interaktionen des Nutzers stark ähnelt ([Sarwar et al. 2001](#)).

Im Kontext moderner Empfehlungssysteme können Nutzer und Items zudem als niedrigdimensionale Vektorrepräsentationen interpretiert werden. Diese Embeddings kodieren latente Präferenz- und Produkteigenschaften, die nicht explizit in den Daten vorliegen, sondern aus Interaktionsmustern gelernt oder aus Produktinformationen abgeleitet werden. Dadurch entsteht eine Verbindung zwischen klassischen Collaborative-Filtering-Ansätzen und Embedding-basierten Matching-Verfahren. Während Collaborative Filtering verhaltensbasierte Ähnlichkeit aus Nutzerinteraktionen modelliert, können Produkt-Embeddings zusätzlich semantische Ähnlichkeiten zwischen Items erfassen ([Wang et al. 2025](#)). Im prototypischen Empfehlungssystem in [Unterabschnitt 5.3](#) werden beide Signale kombiniert: Der Collaborative-Filtering-Score bildet historische Interaktionsmuster ab, während der Embedding-Score semantische Produktnähe ergänzt.

4.6 Optimierung und Evaluierung

Die zuvor beschriebenen Architekturkomponenten bilden die technische Grundlage Embedding-basierter Systeme. Für den produktiven Einsatz reicht es jedoch nicht aus, Daten in Embeddings zu überführen und über einen Vektorindex vergleichbar zu machen. Entscheidend

ist vielmehr, ob die zurückgegebenen Ergebnisse im jeweiligen Anwendungskontext tatsächlich relevant, korrekt und effizient nutzbar sind. Die Qualität eines solchen Systems hängt daher nicht allein vom verwendeten Modell ab, sondern vom Zusammenspiel aus Datenaufbereitung, Textrepräsentation, Chunking-Strategie, Embedding-Modell, Ähnlichkeitsmaß, Vektorindex, Ranking-Logik und gegebenenfalls hybriden Ergänzungen durch lexikalische Verfahren.

Die Optimierung kann auf mehreren Ebenen erfolgen. Bereits die Datenaufbereitung beeinflusst, welche Informationen in den späteren Embeddings enthalten sind. Bei längeren Dokumenten ist insbesondere die Wahl geeigneter Chunking-Strategien relevant, da zu große Segmente spezifische Informationen verwässern können, während zu kleine Segmente wichtigen Kontext verlieren. Auch die Wahl des Embedding-Modells ist anwendungsabhängig: Für semantische Suche und RAG sind Satz- oder Dokument-Embeddings besonders relevant, während Empfehlungssysteme zusätzlich Nutzer- und Item-Repräsentationen benötigen. Auf Retrieval-Ebene können hybride Verfahren eingesetzt werden, die semantische Vektorsuche mit lexikalischen Verfahren, strukturierten Filtern oder nachgelagertem Re-Ranking kombinieren. BM25 ist ein klassisches lexikalisches Ranking-Verfahren, das Dokumente auf Basis gewichteter Begriffsübereinstimmungen bewertet und dabei unter anderem Termhäufigkeit sowie Dokumentlänge berücksichtigt ([Robertson and Zaragoza 2009](#)). In Kombination mit semantischer Vektorsuche können solche Verfahren dazu beitragen, semantische Breite und exakte fachliche Relevanz besser auszubalancieren ([Thakur et al. 2021](#); [Kamath et al. 2024](#)).

Die Evaluierung Embedding-basierter Systeme muss entsprechend anwendungsspezifisch erfolgen. Im Retrieval-Kontext stehen typischerweise Fragen im Vordergrund wie: Werden relevante Dokumente oder Produkte gefunden, erscheinen sie weit oben in der Ergebnisliste und bleibt die Suche auch bei großen Datenmengen effizient? Dafür können Metriken wie Precision, Recall, Mean Reciprocal Rank oder NDCG verwendet werden. Diese Metriken erfassen unterschiedliche Aspekte der Ergebnisqualität, etwa Trefferrelevanz, Vollständigkeit und die Position relevanter Ergebnisse in der Rangliste.

Bei RAG-Systemen ist zusätzlich zu prüfen, ob die generierten Antworten tatsächlich durch die abgerufenen Informationen gestützt sind. Bei Empfehlungssystemen verschiebt sich der Fokus stärker auf Ranking-Qualität, Personalisierung und die Passung der emp-

fohlenen Items zu Nutzerpräferenzen. Neben der reinen Ergebnisqualität sind im Enterprise-Kontext außerdem Effizienz, Skalierbarkeit, Latenz, Aktualisierbarkeit der Datenbasis und Monitoring relevant (Thakur et al. 2021; Karpukhin et al. 2020). Im Rahmen dieser Arbeit wird keine detaillierte quantitative Evaluierung einzelner Modelle oder Systemkonfigurationen durchgeführt. Der Fokus liegt stattdessen auf der konzeptionellen Analyse gemeinsamer Architekturprinzipien Embedding-basierter Enterprise-Systeme. Die betrachteten Use-Cases dienen daher nicht als Benchmark optimierter Produktivsysteme, sondern als prototypische Veranschaulichung der gemeinsamen Verarbeitungsschritte. Eine systematische Evaluation, beispielsweise durch den Vergleich verschiedener Embedding-Modelle, Ranking-Verfahren, Chunking-Strategien oder Indexparameter, wäre ein sinnvoller nächster Schritt für eine produktionsnahe Weiterentwicklung.

4.7 Grenzen von Embeddings

Trotz ihrer vielfältigen Einsatzmöglichkeiten weisen Embeddings mehrere Einschränkungen auf. Eine zentrale Grenze besteht darin, dass semantische Nähe im Vektorraum nicht automatisch fachliche Korrektheit bedeutet. Zwei Begriffe, Dokumente oder Produkte können im Embedding-Raum nahe beieinanderliegen, obwohl sie im konkreten Anwendungskontext nicht austauschbar sind. Dies ist insbesondere bei domänenspezifischen Begriffen, numerischen Angaben oder rechtlich relevanten Informationen problematisch. Ein Beispiel hierfür sind Zahlenwerte: Aus semantischer Sicht können Begriffe wie „fünf“ und „fünfundfünfzig“ ähnliche sprachliche Kontexte besitzen, obwohl sie fachlich völlig unterschiedliche Werte darstellen. Für Retrieval-Prozesse bedeutet dies, dass Embeddings allein nicht immer ausreichen, um exakte numerische Bedingungen zuverlässig abzubilden. In produktiven Systemen sollten semantische Verfahren daher durch strukturierte Filter, Metadaten oder regelbasierte Komponenten ergänzt werden.

Daneben können Embedding-basierte Systeme relevante Informationen übersehen oder irrelevante Treffer zurückgeben, insbesondere wenn eine Anfrage sehr kurz, mehrdeutig oder nicht ausreichend domänenspezifisch formuliert ist. In RAG-Systemen können solche Retrieval-Fehler direkt an ein Sprachmodell weitergegeben werden. Dadurch besteht die Gefahr, dass das Modell eine sprachlich plausible Antwort erzeugt, die jedoch auf unvollständigem oder falschem Kontext basiert. Die Qualität eines RAG-Systems ist daher nicht allein als Qualität des Sprachmodells zu verstehen, sondern als Ergebnis der

gesamten Pipeline aus Datenaufbereitung, Indexierung, Retrieval, Prompt-Kontext und Antwortgenerierung ([Kamath et al. 2024](#)).

Eine gravierende Limitierung ergibt sich aus der Datenrepräsentation. Embeddings komprimieren sprachliche oder strukturierte Informationen in eine feste Vektorrepräsentation. Dabei können spezifische Details verloren gehen, insbesondere wenn längere Dokumente, Produktbeschreibungen oder mehrere Attribute in einem einzigen Vektor zusammengeführt werden. Dieses Problem ist für Enterprise-Anwendungen relevant, da fachlich wichtige Informationen häufig in einzelnen Attributen, Zahlenwerten, Kategorien oder Metadaten enthalten sind. Eine geeignete Segmentierung, die Trennung strukturierter und unstrukturierter Merkmale sowie hybride Such- und Rankingverfahren können diese Einschränkung reduzieren, aber nicht vollständig beseitigen ([Camacho-Collados and Pilehvar 2020](#)).

Problematisch ist zusätzlich die Verzerrung in Trainingsdaten. Da Embedding-Modelle statistische Muster aus großen Textkorpora lernen, können gesellschaftliche, kulturelle oder geschlechtsbezogene Stereotype in den resultierenden Vektorrepräsentationen abgebildet werden. Dies kann insbesondere im Unternehmenskontext kritisch sein, wenn Embeddings in entscheidungsunterstützenden Systemen eingesetzt werden. Die Auswahl geeigneter Modelle sowie die Prüfung auf Bias stellen daher wichtige Anforderungen an produktive Anwendungen dar ([Camacho-Collados and Pilehvar 2020](#)).

Schließlich sind auch technische und organisatorische Aspekte zu berücksichtigen. Moderne Embedding-Modelle und Vektorindizes verursachen Rechenaufwand, Speicherbedarf und Wartungskosten. Zudem müssen Unternehmensdaten häufig besonderen Anforderungen an Datenschutz, Zugriffskontrolle, Aktualität und Nachvollziehbarkeit genügen. Embeddings sind daher nicht als isolierte Lösung zu verstehen, sondern als Bestandteil einer größeren Systemarchitektur, die durch Datenqualität, Evaluierung, Monitoring und geeignete Sicherheitsmechanismen ergänzt werden muss.

4.8 Pipeline Embedding-basierter Enterprise-Systeme

Die in diesem Kapitel beschriebene Architektur zeigt, dass Embedding-basierte Enterprise-Systeme trotz unterschiedlicher Anwendungsziele auf einer gemeinsamen semantischen Verarbeitungsschicht beruhen. Unabhängig davon, ob Embeddings für semantische Suche,

Retrieval-Augmented Generation oder Empfehlungssysteme eingesetzt werden, wiederholen sich zentrale Schritte: Daten werden zunächst aufbereitet, anschließend in semantische Vektorrepräsentationen überführt, in einem Vektorraum gespeichert und über geeignete Ähnlichkeitsmaße miteinander verglichen. Die Unterschiede zwischen den Use-Cases entstehen daher weniger auf Ebene der grundlegenden Embedding-Pipeline, sondern vor allem in der nachgelagerten Nutzung der erzeugten Vektoren.

Diese gemeinsame Grundstruktur ist abstrahiert in [Abschnitt 4](#) dargestellt. Während die semantische Suche die ähnlichsten Dokumente oder Produkte direkt als Treffer zurückgibt, nutzt ein RAG-System die abgerufenen Inhalte zusätzlich als Kontext für ein generatives Sprachmodell. Empfehlungssysteme verwenden Embeddings dagegen, um Nutzer-, Produkt- oder Item-Repräsentationen miteinander zu vergleichen und daraus Rankings oder personalisierte Vorschläge abzuleiten.

Aus Unternehmenssicht bietet diese gemeinsame semantische Verarbeitungsschicht einen wesentlichen Vorteil. Zentrale Komponenten wie Datenaufbereitung, Embedding-Erzeugung etc. müssen nicht für jeden Use-Case vollständig neu implementiert werden. Stattdessen können sie als wiederverwendbare Bausteine verstanden werden, die modular erweitert und an unterschiedliche Anwendungsszenarien angepasst werden. Dadurch lassen sich Entwicklungsaufwand, Systemkomplexität und Infrastrukturkosten reduzieren. Das untermauert wiederum, dass die Performanz der nachgelagerten Anwendungen stark von der Qualität dieser gemeinsamen Basis abhängt.

Abschließend zu diesem Kapitel verdichtet [Tabelle 2](#) diese Zusammenhänge, indem sie die betrachteten Use-Cases den jeweils herausgearbeiteten geeigneten Embedding-Methoden sowie zentralen Nutzen- und Risikodimensionen zuordnet. Dadurch wird deutlich, dass semantische Produktsuche, RAG-basierte Chatbots und Empfehlungssysteme nicht als isolierte Einzelanwendungen zu verstehen sind, sondern als unterschiedliche Spezialisierungen derselben Embedding-basierten Grundarchitektur.

5 Business Use-Cases anhand von Embeddings

Aufbauend auf der in [Abschnitt 4](#) beschriebenen Grundarchitektur wird im Folgenden aufgezeigt, wie Embeddings in konkreten Business-Use-Cases eingesetzt werden können. Hier-

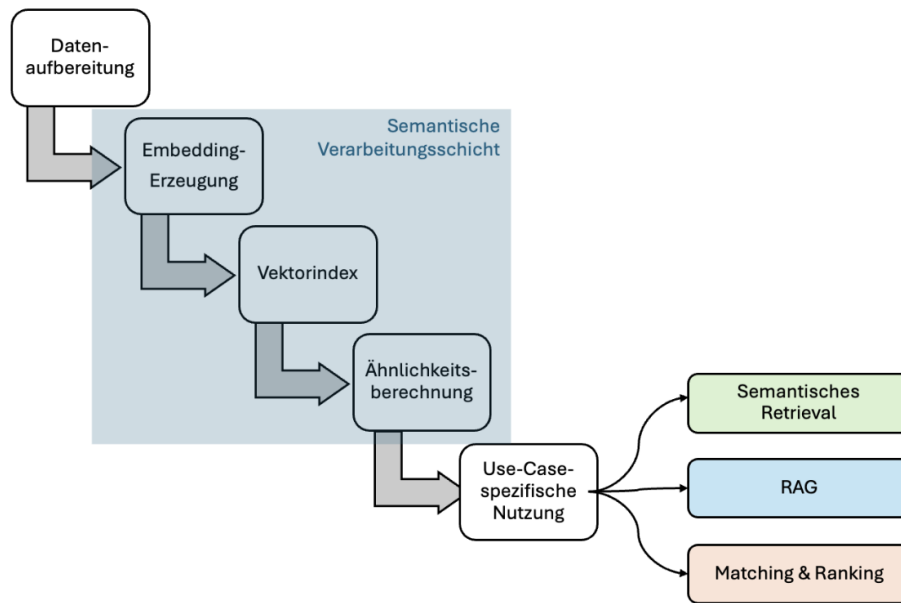


Abbildung 5: Grundarchitektur der Verarbeitung Embedding-basierter Systeme.

Use-Case	Geeignete Embedding-Methode	Vorteil	Hauptrisiko
Semantische Suche	Satz- und Dokument-Embeddings, SBERT-Varianten, domänenspezifische Embeddings	Findet semantisch ähnliche Produkte auch ohne exakte Keyword-Übereinstimmung	Kurze Queries, fehlende lexikalische Präzision und unvollständige Produktattribute
RAG-basierter Chatbot	Satz- und Dokument-Embeddings, SBERT-Varianten	Macht unternehmensspezifisches Wissen natürlichsprachlich nutzbar	Fehlerhaftes Retrieval kann zu plausibel formulierten, aber inhaltlich falschen Antworten führen
Empfehlungssystem	Item- und User-Embeddings, Collaborative Filtering	Kombiniert User-Verhalten und semantische Produktnähe	Cold-Start-Probleme, Datenschutzanforderungen und fehlende Score-Kalibrierung

Tabelle 2: Systematische Einordnung Embedding-basierter Use-Cases im Enterprise-Kontext.

zu werden prototypisch drei Anwendungsszenarien realisiert: semantische Suche, Retrieval-Augmented Generation (RAG) sowie Empfehlungssysteme. Der Fokus liegt dabei auf der exemplarischen Darstellung typischer Anwendungsszenarien sowie deren konzeptioneller Einordnung und weniger auf einer quantitativen Bewertung einzelner Methoden.

Die Implementierung der funktionsfähigen Prototypen erfolgt in Python. Der vollständige Quellcode, die Konfigurationsdateien sowie der evaluierte Datensatz (80 strukturierte Lebensmittelprodukte) sind im zugehörigen GitHub-Repository hinterlegt ¹. Der verwendete Datensatz umfasst 80 exemplarische Lebensmittelprodukte und dient als kontrollierter Proof of Concept zur Demonstration der Architektur. Die Implementierungen sind nicht als optimierte Produktivsysteme zu verstehen, sondern als exemplarische Umsetzungen der zuvor beschriebenen semantischen Verarbeitungspipeline.

5.1 Anwendungsbeispiel: Semantische Produktsuche im Enterprise-Kontext

Die semantische Suche basiert auf dem in [Unterabschnitt 4.3](#) beschriebenen Embedding-Retrieval und stellt einen zentralen Anwendungsfall von Vektorrepräsentationen dar. Sie ist insbesondere im Enterprise-Kontext relevant, da große und heterogene Datenbestände häufig unterschiedliche Terminologien, Schreibweisen und Formulierungen enthalten. Embedding-basierte Suchverfahren können solche Daten semantisch erschließen und dadurch für Nutzer leichter auffindbar und nutzbar machen. Zur Veranschaulichung dieser Funktionsweise wird im Folgenden ein prototypisches Anwendungsbeispiel betrachtet.

5.1.1 Methodik

Für einen Prototypen wurde ein einfacher Vektorindex mit 80 Lebensmittelprodukten erstellt. Beispielhaft ist in [Abbildung 6](#) eines der verwendeten Produkte dargestellt. Diese liegen im JSONL-Format vor und enthalten neben dem Produkttitel auch semantisch relevante Attribute wie Zutaten und Kategorie. Die Produkte wurden mithilfe eines BERT-basierten Embedding-Modells, wie es in [Unterabschnitt 3.3.1](#) beschrieben wurde, in einen semantischen Vektorraum eingebettet. Im Rahmen des Proof of Concept wurde bewusst ein grundlegendes BERT-basiertes Modell verwendet und die gesamten Produkt-

¹Vgl. Repository unter: https://github.com/xenluna/bachelors_thesis_embeddings

beschreibungen (inklusive Titel, Zutaten und Kategorie) unpartitioniert in einen einzigen, globalen Vektor überführt. Diese Vereinfachung dient der nachvollziehbaren Demonstration der vorgestellten Pipeline, stellt jedoch keine optimierte Produktivlösung dar.

```
{
  "id": "p1",
  "title": "Bio Haferdrink Natur",
  "ingredients": ["Wasser", "Hafer", "Salz"],
  "category": "Getraenke"
}
```

Abbildung 6: Beispielhafte Produktrepräsentation im JSONL-Format

Anschließend wurden die Queries „Apfel“ und „Apfelsaft“ an den Vektorindex gestellt. Die resultierenden Top-5-Ergebnisse sind in [Tabelle 3](#) und [Tabelle 4](#) dargestellt. Für die Ähnlichkeitsberechnung zwischen Query-Embedding und Produkt-Embedding wurde ein FAISS-Index vom Typ `IndexFlatIP` verwendet, der das Skalarprodukt zwischen Vektoren berechnet. Da sowohl die Produktvektoren als auch der Query-Vektor vor dem Vergleich L2-normalisiert werden, entspricht dieses Skalarprodukt der Kosinus-Ähnlichkeit. Höhere Werte zeigen somit eine stärkere semantische Nähe zwischen Suchanfrage und Produktrepräsentation im Vektorraum an.

Tabelle 3: Top-5-Ergebnisse der semantischen Suche für die Query „Apfel“

Index	Kosinus-Ähnlichkeit	Produktname	Kategorie
1	0.5329	Avocado	Obst
2	0.4903	Haferflocken Feinblatt	Frühstück
3	0.4734	Basilikum Frisch	Kräuter
4	0.4610	Apfelmus Ungesüßt	Obstkonserven
5	0.4607	Apfel-Birnen Mus	Obstkonserven

Tabelle 4: Top-5-Ergebnisse der semantischen Suche für die Query „Apfelsaft“

Index	Kosinus-Ähnlichkeit	Produktname	Kategorie
1	0.8145	Apfelsaft Naturtrüb	Getränke
2	0.8009	Apfel-Mango Saft	Getränke
3	0.7798	Apfel-Möhren Saft	Getränke
4	0.7473	Apfel-Birnen Saft	Getränke
5	0.7461	Apfel-Blaubeer Smoothie	Getränke

5.1.2 Interpretation der Ergebnisse

Die Resultate in [Tabelle 3](#) und [Tabelle 4](#) veranschaulichen, dass Embedding-basierte Suchverfahren semantische Zusammenhänge über reine Wortübereinstimmungen hinaus erfassen.

sen können, bei kurzen und mehrdeutigen Queries jedoch an Grenzen stoßen. Die Query „Apfel“ zeigt eine zentrale Limitation des prototypischen Systems. Obwohl apfelbezogene Produkte wie „Apfelmus Ungesüßt“ und „Apfel-Birnen Mus“ im Datensatz vorhanden sind, wird mit „Avocado“ zunächst ein Produkt zurückgegeben, das zwar ebenfalls der Kategorie Obst zugeordnet werden kann, aus Nutzersicht jedoch nicht die naheliegendste Antwort auf die Suchanfrage darstellt. Dies deutet darauf hin, dass das verwendete Standardmodell ohne domänenspezifisches Fine-Tuning bei sehr kurzen und unspezifischen Queries nur eingeschränkt zwischen konkreter lexikalischer Relevanz und allgemeiner semantischer Nähe innerhalb der Produktdomäne unterscheiden kann.

Die vergleichsweise niedrigen Scores der Query „Apfel“ im Bereich von etwa 0.46 bis 0.53 veranschaulichen zudem, dass die semantische Übereinstimmung insgesamt schwächer ausfällt als bei der spezifischeren Query „Apfelsaft“. Bei „Apfelsaft“ liegen die Top-Ergebnisse ausschließlich im Bereich apfelbasierter Getränke und erreichen deutlich höhere Kosinus-Ähnlichkeiten zwischen etwa 0.75 und 0.81. Dies spricht dafür, dass spezifischere Suchanfragen mit stärkerem Produktbezug im verwendeten Vektorraum zuverlässiger repräsentiert werden als kurze, mehrdeutige oder allgemeinere Begriffe. Der Unterschied zwischen beiden Suchanfragen lässt sich dadurch erklären, dass „Apfel“ als einzelner Begriff semantisch breiter interpretierbar ist: Das Wort kann als Frucht, Geschmacksrichtung, Zutat oder Bestandteil eines Produktnamens auftreten. Die Query „Apfelsaft“ enthält dagegen bereits einen konkreten Produkttyp und liefert dem Modell damit ein stärkeres semantisches Signal in Richtung Getränkekategorie.

Die Gegenüberstellung beider Queries zeigt somit sowohl das Potenzial als auch die Grenzen von Embedding-basiertem Retrieval. Einerseits können semantisch verwandte Produkte auch ohne reine Schlüsselwortsuche identifiziert werden. Andererseits können bei unscharfen Anfragen Ergebnisse entstehen, die aus Nutzersicht unerwartet wirken. Daraus lässt sich ableiten, dass semantische Nähe im Embedding-Raum nicht notwendigerweise mit nutzerseitiger Suchintention übereinstimmt. Eine reine Embedding-Suche kann insbesondere bei kurzen Produktqueries zu einer Übergewichtung allgemeiner semantischer Kategorien führen. Die Qualität der Suchergebnisse hängt daher wesentlich von der Präzision der Anfrage, der Datenrepräsentation, der Modellwahl und der Domänenanpassung ab.

Die Ergebnisse sind zudem vor dem Hintergrund der vereinfachten Produktrepräsentation zu interpretieren. Da alle Attribute in einem einzigen Produktvektor zusammengeführt wurden, können spezifische Merkmale im Gesamtvektor abgeschwächt werden. Eine Verbesserung der Suchqualität wäre insbesondere durch reichhaltigere Produktbeschreibungen, domänenspezifisches Fine-Tuning, Query-Expansion oder hybride Retrieval-Ansätze möglich, die semantische Embeddings mit lexikalischen Verfahren wie BM25 kombinieren.

Für die Architektur eines produktiven Suchsystems ergibt sich daraus, dass Embedding-Retrieval nicht als Ersatz, sondern als Ergänzung klassischer Suchverfahren verstanden werden sollte. Besonders bei kurzen Produktqueries sind lexikalische Signale, strukturierte Filter und gegebenenfalls ein Re-Ranking notwendig, um semantische Breite und konkrete Produktrelevanz auszubalancieren.

5.2 Anwendungsbeispiel: Enterprise Wissen in Chatbots durch RAG

Der Einsatz von Retrieval-Augmented Generation (RAG) stellt eine Erweiterung der zuvor beschriebenen Embedding-basierten Suche dar, bei der semantisches Retrieval mit generativer Sprachverarbeitung kombiniert wird (vgl. [Unterabschnitt 4.4](#)). Im Enterprise-Kontext bietet dieser Ansatz einen besonderen Mehrwert, da unternehmensspezifisches Wissen nicht nur gefunden, sondern direkt in eine verständliche und kontextbezogene Antwort überführt werden kann. Dies ist insbesondere für interne Wissenssysteme, Produktberatung oder Kundenservice-Anwendungen relevant, bei denen Nutzer nicht nur Dokumente abrufen, sondern konkrete Antworten erhalten möchten.

5.2.1 Methodik

Der zweite Prototyp implementiert einen RAG-basierten Chatbot, der auf den zuvor erstellten Embedding-Index zugreift und Antworten auf Basis der abgerufenen Produktinformationen generiert. Die Retrieval-Komponente des Chatbots folgt dabei derselben Bi-Encoder-Logik wie die zuvor dargestellte semantische Suche. Die Nutzeranfrage wird zunächst als Query-Embedding repräsentiert und mit den bereits gespeicherten Produkt-Embeddings verglichen. Erst die daraus resultierenden Top-5-Treffer werden dem Sprachmodell als Kontext übergeben. Die generative Antwort wird damit auf die zuvor abge-

rufenen Produktinformationen gestützt. Als generatives Sprachmodell wurde im Prototyp gpt-4.1-mini verwendet. Die konkrete Modellwahl dient in dieser Arbeit primär der Demonstration der RAG-Pipeline. Im Fokus steht nicht der Vergleich unterschiedlicher Sprachmodelle, sondern das Zusammenspiel aus Retrieval und Antwortgenerierung. Der System-Prompt definiert das Verhalten des Modells als Unternehmensassistent, der Antworten ausschließlich auf Grundlage der abgerufenen Produktinformationen generieren soll. Die Query-Funktion des Vektorindex wurde dem Sprachmodell als Tool bereitgestellt. Dadurch kann das Modell bei einer Nutzeranfrage relevante Produktinformationen aus dem Index abrufen und diese in die Antwortgenerierung einbeziehen. Zusätzlich werden dem Sprachmodell die letzten Nachrichten des Dialogverlaufs als Kontext übergeben.

Dialogbeispiel: RAG-basierter Produkt-Chatbot

System: Chat gestartet (Tippe „exit“ zum Beenden)

Bot: Hallo! Ich helfe Ihnen bei Fragen zu Produkten.

User: Ich möchte einen Apfelsaft trinken. Welcher steht zur Verfügung?

Bot: Folgende Apfelsäfte stehen zur Verfügung:

1. Apfelsaft Naturtrüb (Zutat: Apfelsaft)
2. Apfel-Mango Saft (Zutaten: Apfel, Mango)
3. Apfel-Möhren Saft (Zutaten: Apfel, Karotte)
4. Apfel-Birnen Saft (Zutaten: Apfel, Birne)
5. Apfel-Blaubeer Smoothie (Zutaten: Apfel, Blaubeeren, Wasser)

Möchten Sie weitere Informationen zu einem dieser Produkte?

User: Ich hätte gerne einen Apfelsaft mit einer exotischen Frucht. Welchen würden Sie mir empfehlen?

Bot: Ich empfehle den *Apfel-Mango Saft*. Dieser enthält Apfel sowie die exotische Frucht Mango. Möchten Sie weitere Informationen dazu?

Abbildung 7: Der beispielhafte Dialog eines RAG-basierten Chatbots

5.2.2 Interpretation der Ergebnisse

Der beispielhafte Dialog in [Abbildung 7](#) demonstriert einen erfolgreichen Fall von Grounding: Die Antwort des Sprachmodells bleibt an die zuvor abgerufenen Produktinformationen gebunden und leitet die Empfehlung „Apfel-Mango Saft“ nachvollziehbar aus dem

Attribut „Mango“ ab. Die Antwort ist damit nicht als freie Generierung zu verstehen, sondern als sprachliche Aufbereitung der im Retrieval gefundenen Produktdaten. Besonders die zweite Nachricht des Nutzers zeigt den Mehrwert der RAG-Struktur: Die unspezifischere Präferenz „exotische Frucht“ wird mit dem abgerufenen Produktattribut „Mango“ verknüpft und in eine konkrete Empfehlung überführt.

Gleichzeitig zeigt das Beispiel, dass die Qualität des Chatbots auf zwei voneinander abhängigen Ebenen bewertet werden muss. Die erste Ebene betrifft das Retrieval: Nur wenn relevante Produktinformationen im Vektorindex korrekt gefunden und dem Sprachmodell als Kontext bereitgestellt werden, kann eine fachlich belastbare Antwort entstehen. Wäre der Apfel-Mango-Saft nicht unter den abgerufenen Top-5-Treffern enthalten gewesen, hätte das Sprachmodell keine ausreichende Grundlage für diese Empfehlung gehabt. Die zweite Ebene betrifft die Generierung: Auch bei grundsätzlich passenden Retrieval-Ergebnissen besteht das Risiko, dass das Sprachmodell Informationen falsch gewichtet, unzulässig ergänzt oder sprachlich plausible, aber nicht ausreichend belegte Aussagen erzeugt. Diese Gefahr wird häufig als Halluzination bezeichnet und ist insbesondere im Enterprise-Kontext kritisch, da fehlerhafte Aussagen zu Produkten, Allergenen, Preisen oder Verfügbarkeiten unmittelbare fachliche Konsequenzen haben können.

Die Qualität des Chatbots ist daher nicht allein als Qualität des Sprachmodells zu interpretieren, sondern als Ergebnis der gesamten Pipeline aus Produktrepräsentation, Vektorindex, Retrieval, Prompt-Kontext und Antwortgenerierung. Daraus leitet sich ab, dass der Chatbot neben der Wahl des LLM insbesondere durch eine Verbesserung der Retrieval- und Kontrollkomponenten optimiert werden müsste. Dazu zählen reichhaltigere Produktbeschreibungen, die getrennte Einbettung einzelner Attribute wie Titel, Kategorie, Zutaten oder Allergene sowie die Kombination semantischer Suche mit lexikalischen Verfahren und strukturierten Filtern. Gerade im Retail-Kontext wäre es sinnvoll, Produktverfügbarkeit, Allergene, Marken, Preise oder Ernährungseigenschaften nicht ausschließlich über Embeddings abzubilden, sondern als Metadatenfilter in die Retrieval-Pipeline einzubinden. Zusätzlich könnte ein Re-Ranking-Schritt eingesetzt werden, um die zunächst gefundenen Top-k-Produkte präziser nach fachlicher Relevanz zu sortieren. Dadurch würde das Sprachmodell zuverlässiger mit passendem Kontext versorgt und das Risiko plausibler formulierter, aber nicht ausreichend belegter Antworten reduziert.

5.3 Anwendungsbeispiel: Retail-Empfehlungssystem

Empfehlungssysteme stellen einen zentralen Anwendungsfall dar, in dem Embeddings zur Modellierung von Beziehungen zwischen Objekten eingesetzt werden (vgl. [Unterabschnitt 4.5](#)). Während in vorherigen Use-Cases vor allem semantische Ähnlichkeiten zwischen Texten im Vordergrund standen, liegt der Fokus hier auf der Abbildung von Nutzerpräferenzen und Produktmerkmalen im gemeinsamen Vektorraum. Aus Unternehmenssicht liegt der Mehrwert insbesondere in der Personalisierung großer Produkt- oder Inhaltsbestände. Durch passende Empfehlungen können relevante Angebote automatisiert priorisiert und Nutzer gezielter angesprochen werden.

5.3.1 Methodik

Ein zentrales Anwendungsbeispiel für Ähnlichkeitszuordnung mittels Embeddings im Retail-Business ist die Generierung personalisierter Angebote für unterschiedliche Kundensegmente. Ziel ist es, auf Basis historischer Interaktionen und semantischer Produktähnlichkeiten solche Produkte zu identifizieren, deren Relevanz für einen gegebenen Nutzer oder ein gegebenes Segment besonders hoch ist. Im Rahmen des prototypischen Anwendungsfalls wurden dieselben Produkte aus den vorherigen Beispielen verwendet und zusätzlich künstlich erzeugte Kundendaten verwendet. Die Kunden wurden dabei mit einer Kauf- und Interaktionshistorie versehen und zusätzlich Segmenten wie *bio_affin*, *proteinreich*, *exotisch*, *vegetarisch*, *italienisch* oder *süßwaren* zugeordnet. Auf Basis historischer Interaktionen wird für jeden Nutzer zunächst ein Präferenzprofil gebildet. Dieses Profil wird anschließend mit den Produktrepräsentationen im Embedding-Raum verglichen, um Produkte mit hoher semantischer Nähe zu identifizieren. [Abbildung 8](#) zeigt exemplarisch ein solches Nutzerprofil. Da sowohl Segmente als auch Interaktionen synthetisch erzeugt wurden, spiegeln die Ergebnisse eher die Annahmen der Datenkonstruktion als reale Kundenpräferenzen wider.

Zur Umsetzung des Collaborative Filterings wurde ein item-basiertes Verfahren gewählt. Hierbei wird aus den Interaktionsdaten aller Nutzer eine User-Item-Matrix konstruiert, in der Einträge die gewichteten Interaktionen (z. B. Kauf, Warenkorb, Ansicht) zwischen Nutzern und Produkten repräsentieren. Auf Basis dieser Matrix wird eine Item-Item-Ähnlichkeitsmatrix berechnet, wobei die Ähnlichkeit zweier Produkte über die Kosinus-

```

{
  "user_id": "u001",
  "segment": "proteinreich",
  "interactions": [
    {"product_id": "p16", "event": "purchase", "weight": 3},
    {"product_id": "p30", "event": "purchase", "weight": 3},
    {"product_id": "p9", "event": "cart", "weight": 2}
  ]
}

```

Abbildung 8: Beispielhafte Nutzerprofile für das Empfehlungssystem

Ähnlichkeit ihrer Interaktionsvektoren bestimmt wird. Produkte gelten somit als ähnlich, wenn sie häufig von denselben Nutzern konsumiert wurden. Für die Generierung von Empfehlungen wird für einen gegebenen Nutzer die Ähnlichkeit zwischen bereits konsumierten Produkten und allen weiteren Produkten aggregiert. Produkte mit hoher kumulierter Ähnlichkeit zu den bereits bekannten Artikeln erhalten dabei einen höheren Score. Bereits konsumierte Produkte werden aus der Ergebnisliste ausgeschlossen. Ergänzend dazu wird ein Embedding-basierter Ansatz integriert, bei dem aus den vom Nutzer konsumierten Produkten ein Profilvektor im Embedding-Raum gebildet wird. Die finale Bewertung eines Produkts ergibt sich aus einer gewichteten Kombination des Collaborative-Filtering-Scores und des Embedding-basierten Ähnlichkeitsscores. Dadurch werden sowohl kollektive Nutzungsmuster als auch semantische Produktähnlichkeiten in der Empfehlung berücksichtigt. Formal wird der finale Score wie folgt berechnet:

$$\text{final_score}(u, i) = \alpha \cdot \text{CF}(u, i) + (1 - \alpha) \cdot \text{Emb}(u, i)$$

Dabei bezeichnet u den Nutzer, i das betrachtete Produkt, $\text{CF}(u, i)$ den verhaltensbasierten Collaborative-Filtering-Score und $\text{Emb}(u, i)$ die semantische Ähnlichkeit zwischen Nutzerprofil und Produktrepräsentation im Embedding-Raum. In der prototypischen Implementierung wurde $\alpha = 0,8$ gewählt, sodass historische Interaktionen stärker gewichtet werden als reine Produktsemantik. Diese Gewichtung ist heuristisch gewählt und dient der Veranschaulichung des kombinierten Ansatzes. Da CF-Score und Embedding-Score in diesem Prototyp nicht auf derselben Skala liegen, ist die Gewichtung nicht als kalibrierte Produktivgewichtung zu verstehen. Die finale Rangfolge ist daher nicht als objektiv optimierte Empfehlung zu verstehen, sondern als heuristische Kombination zweier unterschiedlicher Signale. Ohne Normalisierung, Validierungsdaten oder A/B-Test kann nicht

bestimmt werden, ob $\alpha = 0,8$ tatsächlich eine sinnvolle Gewichtung darstellt.

Tabelle 5: Personalisierte Empfehlungen für Nutzer u001 (Segment: proteinreich)

Rang	Produkt	Kategorie	CF-Score	Emb-Score	Final Score
1	Bio Rind Steak	Fleisch	4.0792	0.8401	3.4314
2	Putenbrust Aufschnitt	Wurstwaren	3.1712	0.8592	2.7088
3	Vollkornreis	Reis	3.1712	0.5469	2.6464
4	Bio Vollmilch	Milchprodukte	1.8236	0.7322	1.6054
5	Thunfisch in Wasser	Fisch	1.4682	0.6609	1.3067

Tabelle 6: Personalisierte Empfehlungen für Nutzer u033 (Segment: süßwaren)

Rang	Produkt	Kategorie	CF-Score	Emb-Score	Final Score
1	Dunkle Schokolade 70%	Süßwaren	5.6866	0.8597	4.7212
2	Buttercroissant	Backwaren	3.8278	0.7699	3.2163
3	Apfelkuchen	Backwaren	2.2751	0.7516	1.9704
4	Apfelstrudel	Backwaren	1.3267	0.7392	1.2092
5	Pecannüsse	Nüsse	0.6781	0.8440	0.7113

Tabelle 7: Personalisierte Empfehlungen für Nutzer u041 (Segment: bio_affin)

Rang	Produkt	Kategorie	CF-Score	Emb-Score	Final Score
1	Bio Walnüsse	Nüsse	3.4215	0.8256	2.9023
2	Bio Vollmilch	Milchprodukte	2.9204	0.5196	2.4403
3	Bio Sojadrink	Getränke	2.1226	0.7535	1.8488
4	Bio Haferdrink Natur	Getränke	2.0157	0.6331	1.7392
5	Bio Lachsfilet	Fisch	1.9807	0.6302	1.7106

5.3.2 Interpretation der Ergebnisse

Die Ergebnisse in [Tabelle 5](#), [Tabelle 6](#) und [Tabelle 7](#) illustrieren, dass der kombinierte Ansatz auf dem konstruierten Datensatz segmenttypisch plausible Empfehlungen erzeugen kann. Aufgrund der künstlich erzeugten Datenbasis ist daraus jedoch keine Aussage über die tatsächliche Empfehlungsqualität in einem produktiven Retail-System ableitbar.

Die Ergebnisse zeigen zugleich unterschiedliche Formen von Relevanz. Im Segment *proteinreich* werden überwiegend Produkte empfohlen, die inhaltlich gut zu einem proteinorientierten Ernährungsprofil passen, etwa Fleisch-, Fisch- und Milchprodukte. Im Segment *süßwaren* erscheinen neben klassischen Süßwaren auch Backwaren wie Apfelkuchen oder Apfelstrudel. Dies ist plausibel, da diese Produkte semantisch mit süßen Lebensmitteln verbunden sind, auch wenn sie nicht der Kategorie Süßwaren zugeordnet sind. Im Segment *bio_affin* werden vor allem Produkte priorisiert, deren Titel explizit den Begriff

„Bio“ enthält. Dies zeigt zugleich eine Grenze des Ansatzes: Ein Teil der Empfehlung kann auf lexikalischen Oberflächenmerkmalen beruhen und nicht ausschließlich auf tiefer semantischer Ähnlichkeit.

Bei der Interpretation der Scores ist zu berücksichtigen, dass der finale Rang vor allem durch den Collaborative-Filtering-Score geprägt wird. Dies zeigt sich beispielsweise bei Nutzer u041: „Bio Vollmilch“ weist mit 0.5196 einen niedrigeren Embedding-Score auf als „Bio Sojadrink“ mit 0.7535, wird aber aufgrund des höheren CF-Scores dennoch höher gerankt. Ähnlich zeigt [Tabelle 6](#), dass „Pecannüsse“ zwar einen hohen Embedding-Score von 0.8440 besitzen, aufgrund des niedrigen CF-Scores jedoch nur auf Rang 5 erscheinen. Der Embedding-Score wirkt im Prototyp daher eher als ergänzendes semantisches Signal, während historische Interaktionsmuster den finalen Rang dominieren. Des Weiteren verdeutlicht [Tabelle 5](#), dass der Embedding-Score bei ähnlichen CF-Werten eine differenzierende Funktion übernehmen kann. „Putenbrust Aufschnitt“ und „Vollkornreis“ weisen denselben CF-Score auf, unterscheiden sich jedoch im Embedding-Score deutlich. Dadurch wird das proteinbezogen plausible Produkt höher gerankt. Dies illustriert, wie semantische Produktähnlichkeit verhaltensbasierte Muster ergänzen kann.

Zusammenfassend lässt sich beobachten, dass die Kombination aus Collaborative Filtering und Embedding-Matching im Prototyp verhaltensbasierte Muster und semantische Produktnähe plausibel zusammenführen kann. Die Ergebnisse zeigen damit die prinzipielle Kombinierbarkeit beider Ansätze, liefern jedoch keinen Nachweis für eine tatsächlich bessere Empfehlungsqualität gegenüber rein kollaborativen oder rein semantischen Verfahren. Auch in diesem Beispiel hängt die Qualität der Empfehlungen stark von der zugrunde liegenden Datenbasis sowie der gewählten Modellierung ab. Für eine belastbare Bewertung wäre daher eine Evaluation auf realen Interaktionsdaten mit geeigneten Ranking-Metriken wie Precision@k, Recall@k oder NDCG erforderlich. In produktiven Systemen könnte ein solcher Ansatz dazu beitragen, relevante Angebote gezielter auszuspielen und dadurch Nutzerinteraktion, Kundenbindung oder Warenkorbwert zu verbessern, sofern die verwendeten Scores zuvor normalisiert, empirisch validiert und datenschutzkonform eingesetzt werden.

5.4 Kritische Einordnung der betrachteten Use-Cases

Die drei betrachteten Use-Cases zeigen, dass Embedding-basierte Enterprise-Systeme trotz unterschiedlicher Anwendungsziele auf einer gemeinsamen semantischen Verarbeitungsschicht beruhen. Semantische Produktsuche, RAG-basierte Chatbots und Empfehlungssysteme nutzen jeweils Vektorrepräsentationen, um Datenobjekte vergleichbar zu machen und daraus Such-, Antwort- oder Rankingentscheidungen abzuleiten. Damit bestätigen die Prototypen die in [Abschnitt 4](#) entwickelte Annahme, dass Embeddings als verbindendes Architekturelement unterschiedlicher Enterprise-KI-Anwendungen verstanden werden können.

Im Zuge der Ausarbeitung wurde deutlich, dass Embeddings keine isolierte Lösung darstellen. Ihre Leistungsfähigkeit hängt nicht allein vom verwendeten Modell ab, sondern von der gesamten Pipeline aus Datenaufbereitung, Datenrepräsentation, Vektorindexierung, Ähnlichkeitsberechnung, Rankinglogik und Evaluation. Semantische Nähe im Vektorraum ist daher nicht automatisch mit fachlicher Korrektheit oder geschäftlicher Relevanz gleichzusetzen. Für produktive Systeme müssen Embeddings deshalb durch strukturierte Attribute, lexikalische Suchkomponenten, Filtermechanismen, Re-Ranking und kontinuierliches Monitoring ergänzt werden.

Für eine mögliche Übertragung in einen SPAR-ICS-nahen Retail-Kontext ist diese Einordnung besonders relevant. Produktdaten im Lebensmitteleinzelhandel bestehen nicht nur aus natürlichsprachlichen Beschreibungen, sondern häufig auch aus strukturierten Attributen wie Kategorien, Zutaten, Allergenen, Marken, Mengenangaben, Preisen oder Verfügbarkeiten. Eine rein textbasierte Vektorrepräsentation kann diese Informationen nur begrenzt abbilden. Daher wäre eine hybride Architektur notwendig, die semantische Embeddings mit strukturierten Produktdaten, Metadatenfiltern und klassischen Suchverfahren kombiniert. Anhanddessen können sowohl semantische Ähnlichkeiten als auch fachlich exakte Anforderungen berücksichtigt werden.

Die Use-Cases unterscheiden sich vor allem darin, an welcher Stelle die zentrale Herausforderung entsteht. Bei semantischer Suche liegt sie in der Balance zwischen breiter semantischer Ähnlichkeit und konkreter Produktrelevanz. Bei RAG-Systemen liegt sie in der Abhängigkeit der Antwortqualität vom vorgelagerten Retrieval. Bei Empfehlungssystemen liegt sie in der Verfügbarkeit geeigneter Interaktionsdaten, der Kalibrierung von

Scores und der datenschutzkonformen Personalisierung. Daraus resultiert, dass dieselbe Embedding-basierte Grundarchitektur je nach Anwendung unterschiedliche Risiken und Optimierungsanforderungen erzeugt.

Aus Sicht von SPAR ICS wäre zudem die Integration in bestehende Unternehmenssysteme entscheidend. Embedding-basierte Anwendungen müssten nicht nur fachlich plausible Ergebnisse liefern, sondern auch mit dynamischen Sortimentsdaten, Aktualisierungen von Produktinformationen, Zugriffskontrollen, Latenzanforderungen und bestehenden Datenpipelines vereinbar sein. Insbesondere bei RAG- oder Suchsystemen ist zu berücksichtigen, dass veraltete oder unvollständige Indexdaten direkt die Ergebnisqualität beeinflussen können. Für Empfehlungssysteme kommen zusätzlich Anforderungen an Datenschutz, Transparenz und den verantwortungsvollen Umgang mit personenbezogenen Interaktionsdaten hinzu.

Insgesamt bilden Embeddings ein leistungsfähiges Bindeglied zwischen Unternehmensdaten und KI-Anwendungen. Ihr praktischer Nutzen entsteht jedoch erst durch die Einbettung in eine robuste, überwachte und domänenspezifisch angepasste Systemarchitektur. Gerade für ein Retail-IT-Unternehmen liegt der zentrale Mehrwert daher nicht allein in einzelnen Embedding-Modellen, sondern in der Fähigkeit, semantische Repräsentationen mit bestehenden Datenstrukturen, Geschäftsprozessen und Qualitätsanforderungen zu verbinden.

Die Prototypen bestätigen somit die architektonische Gemeinsamkeit Embedding-basierter Systeme, zeigen aber zugleich, dass die konkrete Eignung der Methoden stark vom Anwendungskontext abhängt. Satz- und Dokument-Embeddings sind für Suche und RAG geeignet, während Empfehlungssysteme zusätzliche Interaktionsdaten benötigen. Die Forschungsfrage kann daher nicht durch die Wahl eines einzelnen Modells beantwortet werden, sondern durch die Kombination geeigneter Embedding-Methoden mit domänenspezifischer Datenmodellierung und Evaluation.

6 Fazit

6.1 Beantwortung der Forschungsfrage

Die vorliegende Arbeit untersuchte die Forschungsfrage, welche Embedding-basierten Methoden sich zur semantischen Integration unternehmensspezifischen Wissens in KI-Systeme eignen und wie darauf aufbauende Business-Use-Cases anhand einer gemeinsamen Architektur realisiert werden können. Die theoretische Analyse zeigte zunächst, dass klassische Wort-Embeddings wie Word2Vec oder GloVe zwar eine wichtige Grundlage semantischer Repräsentation bilden, für moderne Enterprise-Anwendungen jedoch nur eingeschränkt geeignet sind. Grund dafür ist, dass sie Wörter unabhängig vom jeweiligen Kontext abbilden und dadurch Mehrdeutigkeiten sowie komplexere Satz- oder Dokumentbedeutungen nur begrenzt erfassen können.

Für die in dieser Arbeit betrachteten Anwendungsszenarien sind daher vor allem moderne Satz- und Dokument-Embeddings sowie kontextualisierte Transformer-basierte Repräsentationen relevant. Für semantische Suche eignen sich insbesondere Satz- und Dokument-Embeddings, da Suchanfragen und Produkt- oder Dokumentbeschreibungen in einem gemeinsamen Vektorraum vergleichbar gemacht werden können. Für RAG-basierte Chatbots sind ebenfalls solche Embeddings zentral, da sie die Retrieval-Komponente bilden und damit bestimmen, welche unternehmensspezifischen Informationen dem Sprachmodell als Kontext bereitgestellt werden. Für Empfehlungssysteme kommen zusätzlich Item-, User- und Interaktions-Embeddings infrage, da sie nicht nur textuelle Ähnlichkeiten, sondern auch Nutzerverhalten und Produktbeziehungen modellieren können.

Die prototypischen Implementierungen in [Abschnitt 5](#) zeigen, dass sich auf Basis derselben Grundarchitektur unterschiedliche Business-Use-Cases realisieren lassen. Die semantische Produktsuche nutzt Embeddings, um Produkte auch dann auffindbar zu machen, wenn Suchanfrage und Produktbeschreibung nicht exakt dieselben Begriffe verwenden. Der RAG-basierte Chatbot erweitert dieses Prinzip, indem die gefundenen Produktinformationen als Kontext für eine natürlichsprachliche Antwortgenerierung verwendet werden. Das Empfehlungssystem zeigt schließlich, wie Embeddings mit Collaborative Filtering kombiniert werden können, um sowohl historische Interaktionsmuster als auch semantische Produktnähe in personalisierte Empfehlungen einzubeziehen.

Damit lässt sich die Forschungsfrage dahingehend beantworten, dass nicht eine einzelne Embedding-Methode für alle Enterprise-Szenarien geeignet ist. Vielmehr hängt die Wahl der Methode vom jeweiligen Anwendungskontext ab. Satz- und Dokument-Embeddings eignen sich besonders für semantisches Retrieval und RAG, während Empfehlungssysteme von einer Kombination aus verhaltensbasierten und semantischen Repräsentationen profitieren. Entscheidend ist dabei weniger das einzelne Embedding-Modell als die gesamte Pipeline aus Datenaufbereitung, Embedding-Erzeugung, Vektorindexierung, Ähnlichkeitsberechnung, Ranking und Evaluation.

Die Arbeit zeigt somit auf, dass Embeddings als gemeinsame semantische Verarbeitungsschicht moderner Enterprise-KI-Systeme verstanden werden können. Ihr praktischer Nutzen entsteht jedoch erst durch die Verbindung mit domänenspezifischer Datenmodellierung, hybriden Suchverfahren, strukturierten Filtern, geeigneter Evaluation und kontinuierlichem Monitoring. Für Unternehmen bedeutet dies, dass Embedding-basierte Systeme besonders dann Mehrwert schaffen, wenn sie nicht isoliert eingesetzt, sondern mit bestehenden Produktdaten, Metadaten, Geschäftsprozessen und Qualitätsanforderungen integriert werden.

6.2 Kritische Reflexion und Grenzen der Arbeit

Aus wissenschaftlicher Sicht müssen die Ergebnisse dieser Arbeit jedoch einer kritischen Restriktionsanalyse unterzogen werden. Da im Rahmen der Methodik eine konzeptionelle Analyse anstelle eines großskaligen, empirisch-quantitativen Benchmarks gewählt wurde, sind die Erkenntnisse primär qualitativer Natur. Die Evaluation anhand des erstellten Datensatzes von 80 Lebensmittelprodukten liefert zwar den softwareseitigen Proof-of-Concept, spiegelt jedoch nicht die vollständige Komplexität und das Rauschen einer echten, unbereinigten Enterprise-Datenbasis wider.

Als wesentliche Limitierung dieser Arbeit ist zu nennen, dass die Aussagekraft der Prototypen durch den kontrollierten und konzeptionellen Charakter der Umsetzung begrenzt ist. Der verwendete Datensatz mit 80 exemplarischen Lebensmittelprodukten sowie die prototypischen GitHub-Implementierungen eignen sich zur Veranschaulichung der gemeinsamen Architektur, ersetzen jedoch keine Evaluation auf realen, umfangreichen und heterogenen Unternehmensdaten. Ebenso wurden keine systematischen Benchmarks verschie-

dener Embedding-Modelle, Indexparameter oder Rankingstrategien sowie keine empirischen Messungen zu Retrieval-Präzision, Systemlatenz oder Skalierbarkeit durchgeführt. Die Ergebnisse sind daher als konzeptionelle Plausibilisierung der abgeleiteten Architektur zu verstehen, nicht als Nachweis produktiver Leistungsfähigkeit im SPAR-ICS-Infrastrukturkontext.

Zudem beschränkt sich die Untersuchung primär auf textbasierte Embedding-Verfahren. Da im modernen Retail-Kontext (z. B. der visuellen Produktsuche) multimodale Ansätze (wie CLIP) eine zunehmende Rolle spielen, deckt die hier erarbeitete Architektur diese Facette nur am Rande ab.

6.3 Ausblick und zukünftige Forschungsansätze

Für zukünftige Arbeiten und eine potenzielle Überführung der Prototypen in den operativen Produktivbetrieb ergeben sich daraus drei wesentliche Forschungs- und Entwicklungsansätze. Erstens sollte die rein textbasierte Vektorrepräsentation um Multimodalität erweitert werden. Gerade im Retail-Kontext der Produktsuche verspricht die gemeinsame Einbettung von Bilddaten und Textattributen über Architekturen wie CLIP eine signifikante Steigerung der Relevanz. Zweitens ist das Potenzial von automatisiertem, domänenspezifischem Fine-Tuning (z. B. über kontrastives Lernen auf unternehmenseigenen Klickpfaden) zu untersuchen, um die im engen Vektorraum auftretende Anisotropie allgemeiner Modelle zu brechen. Drittens erfordert das dynamische Umfeld eines Enterprise-Systems eine tiefere Betrachtung von Real-Time-Updates innerhalb der Vektordatenbanken, da minütliche Änderungen von Produktverfügbarkeiten oder Preisen die deterministische Indexierung vor logistische Herausforderungen stellen. Zusammenfassend zeigt diese Arbeit, dass Embeddings das Fundament für intelligente Enterprise-Systeme legen, die tatsächliche Ingenieursleistung der Zukunft jedoch in der Beherrschung und Orchestrierung der gesamten Pipeline liegt.

Literatur

- David M. Blei, Andrew Y. Ng, and Michael I. Jordan. Latent dirichlet allocation. *J. Mach. Learn. Res.*, 3(null):993–1022, March 2003. ISSN 1532-4435.
- Jose Camacho-Collados and Mohammad Taher Pilehvar. Embeddings in natural language processing. In *Proceedings of the 28th International Conference on Computational Linguistics: Tutorial Abstracts*, Barcelona, Spain (Online), December 2020. International Committee for Computational Linguistics.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. URL <https://arxiv.org/abs/1810.04805>.
- Hanna Dünschede. Wenn ki den spickzettel zücht (1/2): Wie funktioniert retrieval augmented generation (rag)? URL <https://blog.ordix.de/wenn-ki-den-spickzettel-zueckt-1>.
- Kawin Ethayarajh. How contextual are contextualized word representations? comparing the geometry of BERT, ELMo, and GPT-2 embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 55–65, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1006. URL <https://aclanthology.org/D19-1006/>.
- J. R. Firth. Applications of general linguistics. *Transactions of the Philological Society*, 56:1–14, 1957. URL <https://api.semanticscholar.org/CorpusID:145333425>.
- Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*. 3rd edition, 2025. URL <https://web.stanford.edu/~jurafsky/slp3/>. Online manuscript released August 24, 2025.
- U. Kamath, K. Keenan, G. Somers, and S. Sorenson. *Large Language Models: A Deep Dive: Bridging Theory and Practice*. Computer Science. Springer Nature Switzerland, 2024. ISBN 9783031656477. URL <https://books.google.at/books?id=kDobEQAAQBAJ>.

- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. Dense passage retrieval for open-domain question answering, 2020. URL <https://arxiv.org/abs/2004.04906>.
- Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with bert, 2020. URL <https://arxiv.org/abs/1901.04085>.
- Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations, 2018. URL <https://arxiv.org/abs/1802.05365>.
- Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://api.semanticscholar.org/CorpusID:160025533>.
- Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks, 2019. URL <https://arxiv.org/abs/1908.10084>.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3:333–389, 09 2009. doi: 10.1561/15000000019.
- Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Prentice Hall Press, USA, 3rd edition, 2009. ISBN 0136042597.
- Gerard Salton, Anita Wong, and Chung-Shu Yang. A vector space model for automatic indexing. *Commun. ACM*, 18:613–620, 1975. URL <https://api.semanticscholar.org/CorpusID:6473756>.
- Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web*, pages 285–295. ACM, 2001. doi: 10.1145/371920.372071.
- Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models, 2021. URL <https://arxiv.org/abs/2104.08663>.
- thumbjinx. Natural language processing: Basic dimension reduction with svd of a co-

occurrence matrix. Cross Validated. URL <https://stats.stackexchange.com/q/390350>. URL:<https://stats.stackexchange.com/q/390350> (version: 2019-02-01).

P. D. Turney and P. Pantel. From frequency to meaning: Vector space models of semantics. *Journal of Artificial Intelligence Research*, 37:141–188, February 2010. ISSN 1076-9757. doi: 10.1613/jair.2934. URL <http://dx.doi.org/10.1613/jair.2934>.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.

Maolin Wang, Xinjian Zhao, Wanyu Wang, Sheng Zhang, Jiansheng Li, Bowen Yu, Binhao Wang, Shucheng Zhou, Dawei Yin, Qing Li, Ruocheng Guo, and Xiangyu Zhao. Embedding in recommender systems: A survey, 2025. URL <https://arxiv.org/abs/2310.18608>.